

STIP: Efficient and Secure Non-Interactive Transformer Inference via Compact Packing

Anonymous Author(s)

ABSTRACT

Secure Transformer Inference (STFI) for LLMs aims to protect both user inputs and model parameters. Fully Homomorphic Encryption (FHE) offers a promising approach for STFI due to its non-interactivity, which eliminates communication overhead. However, FHE-based STFI incurs significant computational costs compared to plaintext inference. Recent advancements have accelerated inference by optimizing packing strategies and reducing the number of rotations. Despite these improvements, several challenges persist, including excessive rotations in ciphertext-ciphertext matrix multiplications (CCMMs), low input/output projection throughput, and expensive maximum/inverse operations, as well as wasted storage slots and inflated ciphertext counts due to sparse packing. To address these issues, we propose STIP, an efficient and secure non-interactive transformer inference framework that incorporates three novel packing strategies: (1) Real-Imaginary Hybrid Packing (RIHP) halves the rotation costs of CCMMs by enabling the simultaneous computation of two output results within the real and imaginary components; (2) Dual-Head Packing (DHP) maps adjacent heads to the real and imaginary components, doubling the throughput of attention projections; and (3) Adaptive Multi-Column Packing (AMCP) packs multiple heads into a single ciphertext, maximizing slot occupancy to reduce the total ciphertext count and thereby enhance computational parallelism. Moreover, for non-linear layers, we employ the Gaussian Kernel instead of Softmax, eliminating the need for maximum value searches and inverse operations, supported by a column-packed RIHP-based L2-norm algorithm. We reformulate Layer Normalization into an inverse-free form by exploiting scale-invariance. Experimental results on an A100 GPU show that STIP achieves 1.6 $\times$, 2.3 $\times$, 4.9 $\times$, and 2.8 $\times$ speedups over Euston (S&P' 26), MOAI (ICLR' 26), THOR (CCS' 25), and Powerformer (ACL' 25), respectively.

CCS CONCEPTS

• Security and privacy $\rightarrow$ Privacy-preserving protocols.

KEYWORDS

Homomorphic encryption, Secure transformer inference

ACM Reference Format:

and Anonymous Author(s). 2018. STIP: Efficient and Secure Non-Interactive Transformer Inference via Compact Packing. In *Proceedings of Make sure*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06...\$15.00
<https://doi.org/XXXXXXX.XXXXXXX>

to enter the correct conference title from your rights confirmation email (Conference acronym 'XX). ACM, New York, NY, USA, 19 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Transformer-based Large Language Models (LLMs) [1], such as BERT [2], GPT [3], and LLaMA [4], have emerged as the cornerstone of modern AI, powering applications from medical diagnosis to financial risk assessment. Leading platforms like OpenAI's ChatGPT and Google's Gemini deploy these models via cloud APIs to deliver scalable services. Consequently, users increasingly rely on these centralized systems to process highly sensitive data [5].

Unfortunately, this cloud-centric model introduces significant privacy risks for both parties [6, 7]. For clients, transmitting sensitive data, such as medical notes or financial records in plaintext, exposes users to the risk of unauthorized access and data leakage [8]. For servers, model parameters constitute critical property, necessitating confidentiality to prevent parameter leakage [9].

Secure transformer inference (STFI) has emerged as a promising paradigm to protect the data privacy of both clients and servers [10]. Such a two-party cryptographic protocol must ensure that the client's input remains confidential from the server while the server's model parameters are protected from the client. Early research on STFI focused on interactive protocols, typically employing Multi-Party Computation (MPC), Oblivious Transfer (OT), Secret Sharing (SS), or hybrid protocol. However, state-of-the-art (SOAT) interactive protocols, such as BOLT [11], SIGMA [12], BumbleBee [13], BLB [14], and SHAFT [15], require transmitting dozens of GB of data between the client and server. This massive communication overhead makes interactive inference difficult to scale in practice. Furthermore, frequent interactions introduce synchronization delays that prevent full exploitation of hardware parallelism.

Consequently, recent works including NEXUS (NDSS' 25) [16], Powerformer (ACL' 25) [17], THOR (CCS' 25) [18], MOAI (ICLR' 26) [19], and Euston (S&P' 26) [20] have pivoted toward non-interactive secure inference based on Fully Homomorphic Encryption (FHE). In this paradigm, the client sends an encrypted request and receives an encrypted response. By executing operations on encrypted data without intermediate communication, these methods enable lower network latency [21] and better hardware acceleration [22, 23].

Currently, MOAI and Euston achieve rapid non-interactive batch inference [24]. By adopting a column-wise packing strategy (packing each column of the input matrix into a distinct ciphertext), they eliminate rotations in both non-linear layers and plaintext-ciphertext matrix multiplications (PCMMs), thereby achieving rapid inference speeds. However, this layout also imposes severe penalties on other critical components.

① **Rotation-intensive CCMMs.** Euston computes the ciphertext-ciphertext matrix multiplication (CCMM) between the matrices $Q \in \mathbb{R}^{m \times d'}$ and $K^T \in \mathbb{R}^{d' \times m}$ requiring $2(m-1)d'$

rotations. By adopting the interleaved packing from [25, 26], MOAI requires $(m - 1)d'$ rotations. For the CCMM of the activation output $S \in \mathbb{R}^{m \times m}$ with $V \in \mathbb{R}^{m \times d'}$, Euston incurs $2(m - 1)d'$ rotations, whereas MOAI requires $2\sqrt{md'} + m$ rotations by adopting [27]. Collectively, these CCMM-induced rotations dominate the total rotation number in both schemes, thereby constituting the primary computational bottleneck.

② **Throughput bottlenecks in linear projections.** The input and output projections within the attention layer are computation-intensive PCMMs. Current schemes perform these projections sequentially for each attention head. This serial execution introduces a linear dependency on the number of heads, causing inference latency to scale with model complexity and thereby constraining the projection throughput.

③ **Severe slot underutilization, low throughput and excessive ciphertexts.** Unlike compact packing schemes including BOLT, Powerformer, and THOR that maximize slot utilization, the sparse packing adopted by MOAI and Euston maps a single column to each ciphertext, thereby leaving the vast majority of slots empty. Under a standard BERT-base setting (token length 128, batch size 32, slots 2^{15}), 87.5% of slots in each ciphertext remain unutilized. This inefficiency has two critical consequences. First, amortized throughput sharply drops in low-batch scenarios, as homomorphic operations incur fixed costs regardless of slot occupancy. This is particularly detrimental for latency-sensitive applications like high-privacy short text queries (e.g., medical symptom diagnosis), where the strict reliance on small batches to avoid accumulation delays [28] leads to massive slot wastage. Second, this sparse packing inflates the ciphertext count, requiring nearly $8\times$ more ciphertexts (2304 vs. 288) to represent the $3 \times (4096 \times 768)$ Q, K, V matrices. This imposes an immense burden on GPU memory.

④ **Limited-scalability Softmax and high-cost Layer Normalization.** The column-wise packing inherently fragments the elements of a single attention row across multiple distinct ciphertexts. Unlike row-wise schemes that can compute the dynamic row maximum to stabilize Softmax, MOAI and Euston are forced to substitute it with a pre-determined constant. This static approach often leads to exponential overflow or underflow, thereby distorting the attention distribution and pushing values outside the convergence interval of homomorphic inverse approximations. Furthermore, their standard Layer Normalization (LN) and Softmax formulations necessitate explicit inverse computations. This operation is computationally expensive, consuming substantial multiplicative depths and requiring extensive key-switching operations.

1.1 Our Contributions

In this paper, we present **STIP**, an efficient and secure non-interactive transformer inference via compact packing. Built upon the residue number system (RNS) variant of Cheon-Kim-Kim-Song (CKKS) scheme [29, 30], STIP addresses the bottlenecks of computation and memory of SOTA schemes.

First, we unlock complex-domain parallelism via imaginary-slot multiplexing. We propose two *Real-Imaginary Hybrid Packing* (RIHP) strategies to minimize the rotation cost of CCMMs (QK^T, SV). The first strategy exploits the diagonal structure of attention scores by encoding the key matrix (K) to simultaneously

compute adjacent output diagonals within the real and imaginary components. The second strategy leverages feature dimension folding by encoding both query (Q) and key (K) matrices, enabling the concurrent accumulation of feature products. Analogously, folding the features of the value matrix (V) permits the parallel computation of different dimensions in the multiplication of S and V . We introduce *Double-Head Packing* (DHP) for input and output projections. By mapping adjacent attention heads to the real and imaginary components, this strategy enables their simultaneous processing, doubling the throughput of input and output projections. Second, to resolve the slot underutilization inherent in the column-wise packing, we introduce *Adaptive Multi-Column Packing* (AMCP). This strategy compacts multiple feature columns into a single ciphertext, ensuring high slot occupancy even at low batch sizes. Third, we redesign the nonlinear function for stability and efficiency. We replace Softmax with a Gaussian Kernel (GK), utilizing the second RIHP strategy to halve the rotation overhead in L2-norm computation. Furthermore, we transform the first LN into an inverse-free variant, exploiting scale-invariance to absorb the scaling factor into the second standard LN, thereby eliminating high-depth and extensive inverse operations. Our contributions are summarized as follows:

① **Compact packing strategies.** Our RIHP and DHP exploit complex-domain parallelism. RIHP reduces the rotation overhead for CCMMs (QK^T, SV) by approximately 50%, and DHP doubles the throughput of input and output projections. Our AMCP achieves highly compact packing to fully utilize ciphertext slots, thereby lowering the ciphertext count and enhancing the parallelism of attention layer algorithms.

② **Efficient and stable nonlinear layers.** We eliminate the dependence of nonlinear layers on maximum and inverse evaluations, which minimizes multiplicative depth and key-switching operations. We substitute Softmax with the GK attention and introduce a column-packed L2-norm algorithm tailored for column-packed matrices. Our second RIHP strategy further reduces the rotation overhead of the L2-norm computation by approximately 50%. We propose an inverse-free LN by exploiting scale-invariance.

③ **Enhanced end-to-end inference efficiency.** Micro-benchmark evaluations on both GPU and CPU demonstrate that STIP outperforms NEXUS and Euston across most computational stages. End-to-end GPU evaluations demonstrate that STIP outperforms Euston, MOAI, THOR, and Powerformer by $1.6\times$, $2.3\times$, $4.9\times$, and $2.8\times$, respectively. The implementation is publicly released at <https://anonymous.4open.science/r/STIP>.

2 PRELIMINARIES

2.1 Transformer Architecture

A transformer layer processes an input embedding matrix through an *attention layer* and a *feed-forward network* (FFN) [1]. In multi-head attentions, the input is projected into several attention heads, whose outputs are concatenated and linearly transformed. Surrounding both the attention layer and the feed-forward network are separate LN operations. After these processing stages, the model produces a logits vector, and the final prediction is obtained by selecting the entry with the highest value.

Attention Layer. For an input embedding matrix $X \in \mathbb{R}^{m \times d}$ (if the batchsize is t , $X \in \mathbb{R}^{tm \times d}$), it is multiplied with the three

Table 1: Frequent notations.

Notation	Description
t	batchsize
m, d, d'	dimensions
H	the number of attention head
q	modulus
N	polynomial degree
S	SIMD slots
H	head number
$\boxplus$	homomorphic addition
$\boxtimes$	homomorphic multiplication
$\boxminus$	homomorphic subtraction
CONJ	homomorphic conjugate
RotL/RotR	left rotation/right rotation
$\mathbf{x} = (x_1, x_2, \dots, x_m)$	vector
$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m]$	matrix
$\llbracket \mathbf{x} \rrbracket$	ciphertext

projection matrices $\mathbf{W}_Q \in \mathbb{R}^{d \times d}$, $\mathbf{W}_K \in \mathbb{R}^{d \times d}$, and $\mathbf{W}_V \in \mathbb{R}^{d \times d}$ to obtain a query matrix $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, a key matrix $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, and a value matrix $\mathbf{V} = \mathbf{X}\mathbf{W}_V$. Under the multi-head attention setting, $\mathbf{X}$ is multiplied with the $d' = d/H$ columns of each parameter matrix that correspond to the current attention head.

The attention is calculated using the formula:

$$\text{ATTENTION}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{SOFTMAX}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d})\mathbf{V} \quad (1)$$

The GK attention is calculated using the formula [31, 32]:

$$\text{ATTENTION}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = S(\mathbf{Q}, \mathbf{K})\mathbf{V} \quad (2)$$

$$S(\mathbf{Q}, \mathbf{K})_{ij} = \exp\left(-\frac{1}{2\sqrt{d}}\|\mathbf{Q}_{i,:} - \mathbf{K}_{j,:}\|_2^2\right), i, j \in [m],$$

where $\|\cdot\|_2$ is L2-norm, $\mathbf{Q}_{i,:}$ and $\mathbf{K}_{j,:}$ mean the i -th and j -th row of $\mathbf{Q}$ and $\mathbf{K}$, and $\exp$ is the exponential operation.

LN. For an input $\mathbf{x} \in \mathbb{R}^n$, LN computes the mean μ and standard deviation σ .

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i, \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2} \quad (3)$$

The normalized output $\mathbf{y}$ is computed by scaling with a learnable parameter $\gamma \in \mathbb{R}^n$ and shifting with a learnable parameter $\beta \in \mathbb{R}^n$.

$$\mathbf{y} = \gamma \cdot \frac{\mathbf{x} - \mu}{\sigma} + \beta \quad (4)$$

Feed-Forward Network. A FFN consists of two linear transformations with an activation function (GELU/ReLU) applied in between. To enable scale propagation within the FFN module, the ReLU activation is employed.

$$\begin{aligned} \text{FFN}(\mathbf{X}) &= \text{ReLU}(\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 \\ &= \max(0, \mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 \end{aligned} \quad (5)$$

2.2 Fully Homomorphic Encryption

RNS-CKKS [30] enables approximate homomorphic computation over the polynomial ring $R_Q = \mathbb{Z}_Q[x]/(x^N + 1)$, where N is a power of two, $Q = \prod_{i=0}^L q_i$ consists of distinct primes q_i , and L is the multiplicative depth. In this leveled scheme, a ciphertext

at level k is associated with modulus $Q_k = \prod_{i=0}^k q_i$. The noise is managed via rescaling, which reduces the modulus by the last prime q_i . For deep circuits, bootstrapping consumes K levels to refresh the ciphertext, leaving $L - K$ levels available for subsequent operations. RNS-CKKS supports Single Instruction Multiple Data (SIMD) by packing vectors via the mapping $\mathbb{C}^{N/2} \rightarrow R_Q$, enabling parallel arithmetic on $S = N/2$ slots.

We summarize below the homomorphic operations employed in this paper:

Homomorphic Addition. Homomorphic addition allows the computation of the sum of two encrypted vectors. Given two ciphertexts $\llbracket \mathbf{a} \rrbracket$ and $\llbracket \mathbf{b} \rrbracket$, the homomorphic addition is $\llbracket \mathbf{a} \rrbracket \boxplus \llbracket \mathbf{b} \rrbracket$.

Homomorphic Multiplication. Homomorphic multiplication computes the element-wise product of two encrypted vectors. Given two ciphertexts $\llbracket \mathbf{a} \rrbracket$ and $\llbracket \mathbf{b} \rrbracket$, the homomorphic multiplication is $\llbracket \mathbf{a} \rrbracket \boxtimes \llbracket \mathbf{b} \rrbracket$. It consumes one level of multiplicative depth, requiring a rescaling operation to manage the ciphertext scale and a relinearization step to recover the ciphertext size.

Homomorphic Rotation. Since CKKS encrypts vectors using SIMD packing, homomorphic rotations are required to enable interactions between elements at different positions.

① Left rotation: $\text{RotL}_k(\llbracket \mathbf{a} \rrbracket)$ shifts the vector $\llbracket \mathbf{a} \rrbracket$ to the left by k positions. If $\llbracket \mathbf{a} \rrbracket = (a_0, a_1, \dots, a_{N-1})$, the result corresponds to an encryption of $(a_k, \dots, a_{N-1}, a_0, \dots, a_{k-1})$.

② Right rotation: $\text{RotR}_k(\llbracket \mathbf{a} \rrbracket)$ shifts the vector $\llbracket \mathbf{a} \rrbracket$ to the right by k positions.

The rotation is a key primitive for summing elements. Moreover, the rotation is computationally expensive because it involves the key-switching operation.

Homomorphic Conjugate. Given a ciphertext $\llbracket \mathbf{a} \rrbracket$ encrypting a complex vector $\mathbf{a} = \mathbf{x} + i\mathbf{y}$, the homomorphic conjugate computes the complex conjugate of each element, denoted as $\text{CONJ}(\llbracket \mathbf{a} \rrbracket) = \llbracket \mathbf{x} - i\mathbf{y} \rrbracket$. This operation is computationally expensive as it requires key-switching.

Homomorphic Sign Function. To evaluate the non-linear sign function, we employ a composite polynomial approximation using basis functions f and g . For a precision boundary $2^{-\alpha}$, the function is defined as [33, 34]:

$$\text{SGN}(x) = f^{d_f}(g^{d_g}(x)) = \begin{cases} -1 & -1 \leq x \leq -2^{-\alpha} \\ 0 & x = 0 \\ 1 & 2^{-\alpha} \leq x \leq 1 \end{cases} \quad (6)$$

For comparison operations (e.g., ReLU), the sign function requires inputs within $[-1, 1]$. To ensure this, we normalize the input vector $\mathbf{a}$ using the scaling factor $\Delta = \max\{|a_{\max}|, |a_{\min}|\}$ and evaluate $\text{SGN}(\mathbf{a}/\Delta)$.

Following NEXUS, we utilize degree-9 basis polynomials (f and g) with parameters $d_f = d_g = 2$ and $\alpha = 20$.

3 OVERVIEW

3.1 System Model and Threat Model

Following prior works [16–20], we consider a secure transformer inference scenario where a client transmits the encrypted input $\mathbf{X} \in \mathbb{R}^{tm \times d}$ to a server, and the server generates responses $\mathcal{M}(\mathbf{X})$ with the model parameter $\mathcal{M}$. We adopt a semi-honest security

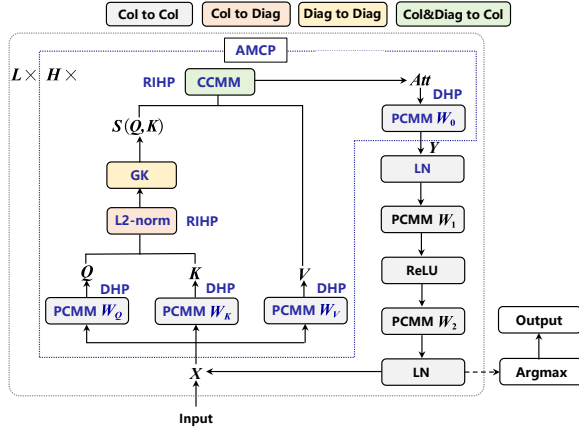


Figure 1: A simple workflow of STIP. The improvements made by STIP are highlighted in blue.

model, assuming both parties adhere to the protocol but the server attempts to reconstruct X and the client attempts to reconstruct M . Our system guarantees: (1) User input privacy, (2) Server model privacy, and (3) Inference correctness.

3.2 Workflow

As depicted in Figure 1, STIP orchestrates a dynamic transition of packing methods between layers.

Input Projection (Col→Col). The workflow initiates with the column-packed (Col) input X . We employ PCMM interactions with weight matrices ($W_{Q,K,V}$) to generate query, key, and value matrices while maintaining the column layout. To accelerate this phase, DHP pairs adjacent heads into real and imaginary components, effectively doubling the throughput. Notably, these PCMM operations are executed without any rotations (cf. Section 4.2).

Attention Mechanism (Col→Diag→Diag). The workflow transitions to a diagonal-packed (Diag) layout to efficiently handle the attention mechanism. We design a column-packed L2-norm algorithm that converts Q and K from column to diagonal packing, utilizing the RIHP to halve the numbers of rotations (cf. Section 4.1 and 5.1). Subsequently, the output of the L2-norm computation is processed by a GK function. This rotation-free GK layer yields the diagonal-packed attention score matrix $S(Q, K)$ (cf. Section 5.1).

Attention Output (Col&Diag→Col). This stage generates the column-packed attention Att by aggregating the diagonal-packed scores $S(Q, K)$ with the column-packed value matrix V . The RIHP strategy and Baby-Step Giant-Step (BSGS) algorithm are employed to minimize rotation overheads in this CCMM (cf. Appendix C).

Optimization across Attention Layer. To achieve the high-parallelism attention layer, we employ AMCP across the above three stages. This strategy aggregates columns from distinct heads to maximize multi-head parallelism for all algorithms within the attention layer and effectively reduces the storage overhead of intermediate ciphertexts (cf. Section 4.3).

Output Projection and Feed-Forward Network (Col → Col). The final stage processes the attention Att through the output projection and FFN via PCMMs ($W_{0,1,2}$), where DHP doubles the

Table 2: Amortized number of key-switching operations required for computing attention scores $\{Q^{(h)}(K^{(h)})^T\}_{h \in [H]}$ per batch, where $Q^{(h)}, K^{(h)} \in \mathbb{R}^{m \times d'}$. Parameters correspond to the BERT-base model ($m = 128, d = 768, d' = 64, H = 12$) with $N = 2^{16}$ and $t = N/2m = 256$. We set $a = 64$ and $c = 16$, consistent with Powerformer and THOR, respectively. “+” indicates the first RIHP. “ $\ddagger$ ” indicates the second RIHP.

Scheme	Key switch	
	Equation	Number
Euston	$(m-1)d'H/(N/4m)$	762
THOR	$\frac{m}{2c}(\frac{m}{2} + 7) + \frac{m}{8}(\log c + 3) + \frac{m}{2}$	460
Powerformer	$\approx (17 + 14\sqrt{2})\sqrt{a} + 3a$	486
MOAI	$(m-1)d'H/(N/2m)$	381
STIP [†]	$((d' + 1)m/2)H/(N/2m)$	195
STIP [‡]	$((m-1)(d'/2) + m)H/(N/2m)$	197

throughput of the output projection (cf. Section 4.2). These PCMM operations are also executed without any rotations. Within this block, the first LN is reformulated to eliminate inverse operations, while the second follows the standard protocol. Crucially, both LN layers involve no rotation operations, and the final result retains column packing (cf. Section 5.2).

4 EFFICIENT MATRIX MULTIPLICATION AND COMPACT PACKING STRATEGY

4.1 Real-Imaginary Hybrid Packing

For computing the attention score QK^T , the SOTA CCMM in MOAI evaluates m diagonals individually: $\llbracket P_r \rrbracket = \sum_{j=0}^{d'-1} \llbracket q_j \rrbracket \boxtimes \text{RotL}_r(\llbracket k_j \rrbracket)$ for $r \in [0, m-1]$, where q_j and k_j denote the j -th feature column of the query and key matrices. However, this method still incurs substantial overheads due to the extensive number of rotations (see Table 2).

To decrease the overhead of computing the attention score, we introduce two RIHP strategies. We only present the first strategy in this section. The second strategy is described in Appendix C, and the details of the second strategy applied to the SV computation are also described in Appendix C. Algorithm 1 presents the pseudocode of the first strategy, and Figure 2 illustrates an example of the first strategy, where we adopt the interleaved batching in [19, 25, 26] for packing multi-batch inputs, which enables efficient intra-batch rotations. The pipeline comprises three stages:

Phase 1: Input Projection with Hybrid Key Matrix. This phase generates the encrypted query and value matrices, while deriving a encrypted hybrid key matrix.

Key Innovation: We introduce a hybrid packing format for the key matrix K , denoted as $\llbracket \hat{K} \rrbracket = [\llbracket \hat{k}_0 \rrbracket, \dots, \llbracket \hat{k}_{d'-1} \rrbracket]$, where each slot encodes original data in the real part and rotated data in the imaginary part: $\hat{k}_j = k_j + i \cdot \text{RotL}_{m/2}(k_j)$. This enables the halved-rotation attention mechanism in **Phase 2**. The implementation strategy distinguishes between the first layer and subsequent layers:

First Layer: Given the input $X \in \mathbb{R}^{m \times d}$, the client encrypts the column-wise input $\llbracket X \rrbracket = [\llbracket x_0 \rrbracket, \dots, \llbracket x_{d-1} \rrbracket]$ for the query and value

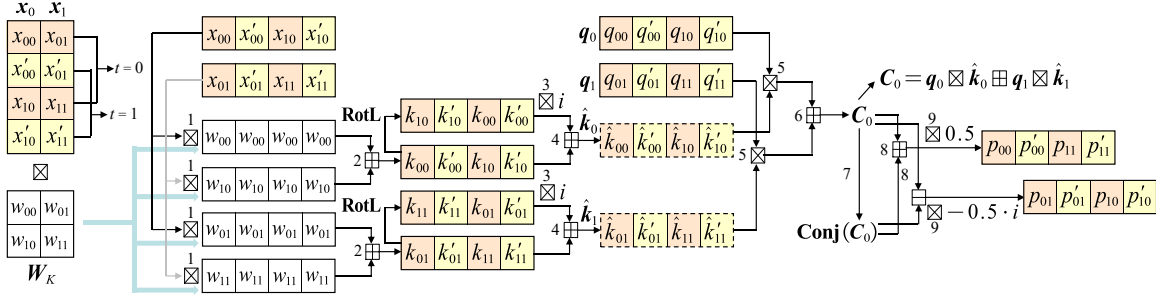


Figure 2: An example of the first RHIP strategy. We set dimensions $m = d = d' = 2$, batch size $t = 2$, and slots $S = 4$. The dashed box indicates the utilization of the imaginary component. “Conj” indicates the conjugate. Under the interleaved batching, the rotational step size is $\phi \cdot \varphi$, where ϕ is the original step size and $\varphi = S/m$.

Algorithm 1 The First RIHP Strategy

$$\begin{aligned}
& \mathbf{1: Input:} \text{ Input matrix } X = \{x_l\}_{l=0}^{d-1}, \text{ Weights } W_O = \{w_{l,j}^O\}_{l \in [d], j \in [d']}, O \in \{Q, K, V\}, o \in \{q, k, v\}. \\
& \mathbf{2: Output:} \text{ Encrypted diagonals of } QK^\top: \{\{P_r\}_{r=0}^{m-1}\}. \\
& \mathbf{3: if} \text{ Layer} = 0 \text{ then} \\
& \quad \mathbf{4: for} \text{ } l = 0, 1, \dots, d-1 \text{ do} \\
& \quad \quad \hat{x}_l \leftarrow x_l + i \cdot \text{RotL}_{m/2}(x_l) \\
& \quad \mathbf{5: end for} \\
& \quad \mathbf{6: for} \text{ } j = 0, 1, \dots, d' - 1 \text{ do} \\
& \quad \quad [\hat{k}_j] \leftarrow \sum_{l=0}^{d-1} w_{l,j}^K \cdot [\hat{x}_l] \\
& \quad \quad [q_j] \leftarrow \sum_{l=0}^{d-1} w_{l,j}^Q \cdot [x_l] \\
& \quad \quad [v_j] \leftarrow \sum_{l=0}^{d-1} w_{l,j}^V \cdot [x_l] \\
& \quad \mathbf{7: end for} \\
& \quad \mathbf{8: else} \\
& \quad \quad [o_j] \leftarrow \sum_{l=0}^{d-1} w_{l,j}^O \cdot [x_l] \\
& \quad \quad [\hat{k}_j] \leftarrow [k_j] \boxplus i \cdot \text{RotL}_{m/2}([k_j]) \\
& \quad \mathbf{9: end if} \\
& \quad \mathbf{10: for} \text{ } r = 0, 1, \dots, m/2 - 1 \text{ do} \\
& \quad \quad [C_r] \leftarrow \sum_{j=0}^{d'-1} ([q_j] \boxtimes \text{RotL}_r([\hat{k}_j])) \\
& \quad \quad [P_r] \leftarrow \frac{1}{2} \boxtimes ([C_r] \boxplus \text{Conj}([C_r])) \\
& \quad \quad [P_{r+m/2}] \leftarrow -\frac{i}{2} \boxtimes ([C_r] \boxminus \text{Conj}([C_r])) \\
& \quad \mathbf{11: end for} \\
& \mathbf{12: return} \{\{P_r\}_{r=0}^{m-1}\}
\end{aligned}$$

matrices. For the key matrix, the client constructs a hybrid plaintext $\hat{X} = X + i \cdot \text{Rot}_{L_{m/2}}(X)$, yielding $[\hat{X}] = [[\hat{x}_0], \dots, [\hat{x}_{d-1}]]$. The client sends $[\hat{X}]$ and $[X]$ to the server.

Upon receiving these ciphertexts, the server computes the input projection via PCMM with plaintext weights. Let $\mathbf{W}_O \in \mathbb{R}^{d \times d'}$ ($\mathbf{O} \in \{\mathbf{Q}, \mathbf{K}, \mathbf{V}\}$) be the weight matrix with $w_{i,j}^O$ representing the weight at row i and column j . Specifically, the j -th column of the hybrid key matrix $\llbracket \hat{\mathbf{K}} \rrbracket$ is:

$$\llbracket \hat{\mathbf{k}}_j \rrbracket = \sum_{l=0}^{d-1} w_{l,j}^K \cdot \llbracket \hat{\mathbf{x}}_l \rrbracket. \quad (7)$$

Similarly, the query and value projections, $\llbracket Q \rrbracket$ and $\llbracket V \rrbracket$, are obtained by applying the weight matrices W_Q and W_V to the encrypted input $\llbracket X \rrbracket$.

Subsequent Layers: For subsequent layers, the input $[\mathbf{X}]$ is the encrypted output of the previous transformer layer. The column-wise encrypted packing of $\mathbf{K}$ is represented as $[\mathbf{K}] = [[\mathbf{k}_0], [\mathbf{k}_1], \dots, [\mathbf{k}_{d'-1}]]$. We compute the standard key vectors $[\mathbf{k}_j]$:

$$\llbracket \mathbf{k}_j \rrbracket = \sum_{l=0}^{d-1} w_{l,j}^K \cdot \llbracket \mathbf{x}_l \rrbracket \quad (8)$$

Then, to enable RIHP strategy, we apply a homomorphic rotation to construct the hybrid key matrix $\llbracket \hat{K} \rrbracket$:

$$[\hat{\mathbf{k}}_j] = [\mathbf{k}_j] \boxplus i \cdot \text{RotL}_{m/2}([\mathbf{k}_j]) \quad (9)$$

Phase2: Halved-Rotation CCMM. We propose a compact CCMM to compute the attention scores $P = QK^\top$.

Key Innovation: Instead of computing m separate diagonals, we leverage the complex nature of CKKS to pack two diagonals into one ciphertext.

For $r \in \{0, \dots, m/2 - 1\}$, the r -th hybrid ciphertext $\llbracket C_r \rrbracket$ is computed as:

$$\llbracket C_r \rrbracket = \sum_{j=0}^{d'-1} \llbracket \mathbf{q}_j \rrbracket \boxtimes \text{RoTL}_r(\llbracket \hat{\mathbf{k}}_j \rrbracket) \quad (10)$$

Due to the linearity of rotation and the hybrid structure of $\llbracket \hat{\mathbf{K}} \rrbracket$, $\llbracket C_r \rrbracket$ simultaneously stores the r -th real diagonal and the $(r + m/2)$ -th imaginary diagonal:

$$\llbracket C_r \rrbracket = \llbracket P_r \rrbracket \boxplus i \cdot \llbracket P_{r+m/2} \rrbracket \quad (11)$$

Complexity Gain: This technique requires only $md'/2$ multiplications and $(m/2 - 1)d'$ rotations, reducing the computational cost by **50%** compared to the MOAI approach.

Phase 3: Conjugate Unpacking. Since the subsequent Softmax function is non-linear and requires pure real inputs, we must separate the packed diagonals. We utilize the homomorphic conjugate operation (CONJ) for efficient extraction:

$$\llbracket P_r \rrbracket = 0.5 \boxtimes (\llbracket C_r \rrbracket \boxplus \text{CONJ}(\llbracket C_r \rrbracket)) \quad (12)$$

$$\llbracket P_{r+m/2} \rrbracket = -0.5i \boxtimes (\llbracket C_r \rrbracket \boxminus \text{CONJ}(\llbracket C_r \rrbracket)) \quad (13)$$

The correctness proof of the first RIHP strategy is provided in Appendix D.

4.2 Double-Head Packing

For input and output projections in the attention layer, MOAI and Euston utilize column packing to achieve rotation-free PCMMs.

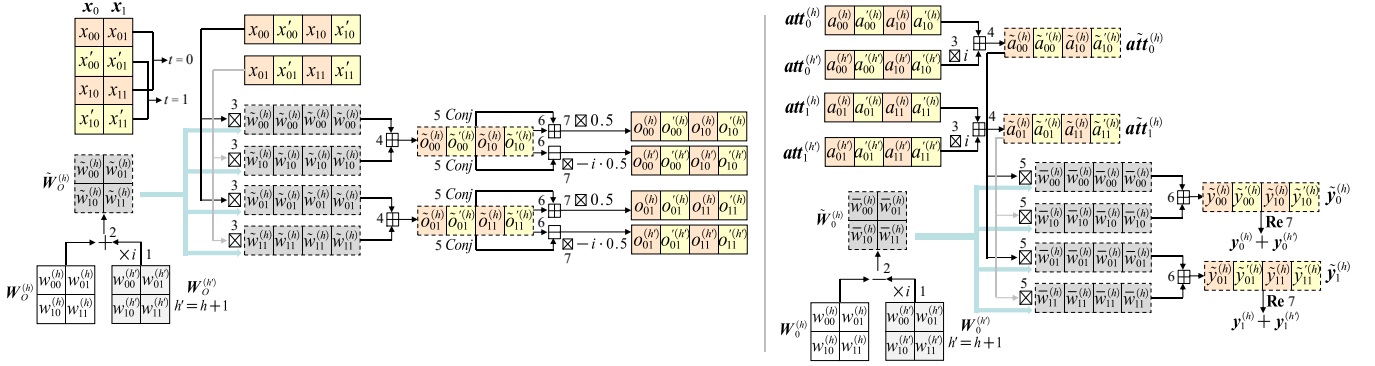


Figure 3: Examples of DHP. The left side is the input projection, and the right side is the output projection. We set dimensions $m = d = d' = 2$, batch size $t = 2$, and slots $S = 4$. The dashed box indicates the utilization of the imaginary component. “Re” denotes the extraction of the real part.

Table 3: Amortized number of key-switching operations required for computing input projections $\{XW_O^{(h)}\}_{h \in [H]}$ per batch, where $X \in \mathbb{R}^{m \times d}$, $W_O^{(h)} \in \mathbb{R}^{d \times d'}$. Parameters are identical to Table 2. “Throughput” refers to the number of heads processed for each input projection.

Scheme	Key switch		Throughput
	Equation	Number	
NEXUS	$2md/(N/2m)$	768	1×
THOR	$\frac{m}{2} + 3(H-2)\frac{d'}{c} + \frac{3d'}{c}$	192	1×
Powerformer	$\approx \sqrt{\frac{5}{9}} \cdot 2\sqrt{\frac{md(3d)}{N/2}}$	122	1×
Euston/MOAI	0	0	1×
STIP	0	0	2×

However, both schemes process a single attention head per ciphertext, limiting potential throughput.

To enhance the computational parallelism of the multi-head attention projection, we introduce a DHP strategy. By exploiting the complex slot of the CKKS, we process two attention heads (h and $h+1$) simultaneously within one ciphertext. This strategy is applied to the input projection ($X \rightarrow Q, K, V$) and the output projection ($Att \rightarrow Y$). Algorithm 2 presents the pseudocode. Figure 3 illustrates an example of DHP. The pipeline comprises four stages:

Phase 1: Offline Weight Pre-coding. We pair adjacent attention heads ($h, h+1$) for $h \in \{0, 2, \dots, H-2\}$. Let $W_O^{(h)}$ be the real-valued weights ($O \in \{Q, K, V\}$). For the input projection, we construct complex weight matrices $\tilde{W}_O^{(h)}$:

$$\tilde{W}_O^{(h)} = W_O^{(h)} + i \cdot W_O^{(h+1)} \quad (14)$$

For the output projection, we construct the weights using a conjugate structure to facilitate efficient aggregation:

$$\tilde{W}_O^{(h)} = W_O^{(h)} - i \cdot W_O^{(h+1)} \quad (15)$$

This preprocessing is performed offline and incurs zero online computational overhead.

Phase 2: Parallel Input Projection. For the column-wise input $[X] = [[x_0], \dots, [x_{d-1}]]$, we compute the projections $[\tilde{O}^{(h)}] = [[\tilde{o}_0^{(h)}], [\tilde{o}_1^{(h)}], \dots, [\tilde{o}_{d'-1}^{(h)}]]$ ($o \in \{q, k, v\}$) for both heads in a single PCMM with the pre-coded weights $\tilde{W}_O^{(h)} \in \tilde{W}_O^{(h)}$:

$$[\tilde{o}_j^{(h)}] = \sum_{l=0}^{d'-1} \tilde{w}_{l,j}^{(h)} \cdot [x_l] \quad (16)$$

Due to the linearity of the operation, the resulting ciphertext $[\tilde{O}^{(h)}]$ naturally encodes the projections for head h in the real part and head $h+1$ in the imaginary part:

$$\tilde{O}^{(h)} = XW_{O,h} + i \cdot XW_{O,h+1} \quad (17)$$

This step effectively doubles the throughput of PCMM.

Phase 3: Unpacking for Attention Mechanism. To prevent cross-term interference arising from the attention score calculation ($QK^\top$) on complex-packed data, we unpack the heads prior to the attention block. Using the homomorphic conjugate operation (CONJ), we extract the individual heads:

$$[O^{(h)}] = 0.5 \boxtimes ([\tilde{O}^{(h)}] \boxplus \text{CONJ}([\tilde{O}^{(h)}])) \quad (18)$$

$$[O^{(h+1)}] = -0.5i \boxtimes ([\tilde{O}^{(h)}] \boxminus \text{CONJ}([\tilde{O}^{(h)}])) \quad (19)$$

Following unpacking, the attention mechanism is applied to obtain the output $[Att^{(h)}] = [[att_0^{(h)}], [att_1^{(h)}], \dots, [att_{d'-1}^{(h)}]]$ and $[Att^{(h+1)}] = [[att_0^{(h+1)}], [att_1^{(h+1)}], \dots, [att_{d'-1}^{(h+1)}]]$.

Phase 4: Parallel Output Projection. After computing the attention outputs for all H heads, we aggregate them into $H/2$ complex pairs. For each pair index $h \in \{0, 2, \dots, H-2\}$, we pack the outputs of head h and $h+1$ into a complex ciphertext:

$$[\tilde{Att}^{(h)}] = [Att^{(h)}] \boxplus i \boxtimes [Att^{(h+1)}] \\ = [[\tilde{att}_0^{(h)}], [\tilde{att}_1^{(h)}], \dots, [\tilde{att}_{d'-1}^{(h)}]] \quad (20)$$

The final output Y is the sum of projections from all heads:

$$Y = \sum_{h=0}^{H-1} Att^{(h)} W_0^{(h)} = [\sum_{h=0}^{H-1} y_0^{(h)}, \dots, \sum_{h=0}^{H-1} y_{d-1}^{(h)}] \quad (21)$$

We compute Y by aggregating the products of the complex pairs and their corresponding pre-coded conjugate weights $\tilde{W}_0^{(h)} = W_0^{(h)} - i \cdot W_0^{(h+1)}$:

$$\tilde{Y} = \sum_h (\tilde{Att}^{(h)} \times \tilde{W}_0^{(h)}), h \in \{0, 2, \dots, H-2\} \quad (22)$$

Algorithm 2 Double-Head Packing

```

1: Input: Input  $\{\llbracket x_l \rrbracket\}_{l=0}^{d-1}$ , Weights  $\{W_O^{(h)}, W_0^{(h)}\}_{h=0}^{H-1}$ ,  $O \in \{Q, K, V\}$ .
2: Output: Encrypted output  $\llbracket Y \rrbracket$ .
3: for  $h = 0, 2, \dots, H-2$  do
4:    $\tilde{W}_O^{(h)} \leftarrow W_O^{(h)} + i \cdot W_O^{(h+1)}, \tilde{W}_0^{(h)} = \{\tilde{w}_{l,j}^{(h)}\}_{l \in [d], j \in [d']}$ 
      $\tilde{W}_0^{(h)} \leftarrow W_0^{(h)} - i \cdot W_0^{(h+1)}, \tilde{W}_0^{(h)} = \{\tilde{w}_{l,j}^{(h)}\}_{l \in [d'], j \in [d]}$ 
5: end for
6:  $\llbracket \tilde{Y} \rrbracket \leftarrow 0$ 
7: for  $h = 0, 2, \dots, H-2$  do
8:    $\llbracket \tilde{o}_j^{(h)} \rrbracket \leftarrow \sum_{l=0}^{d-1} \tilde{w}_{l,j}^{(h)} \cdot \llbracket x_l \rrbracket, o \in \{q, k, v\}$ 
9:    $\llbracket o^{(h)} \rrbracket \leftarrow 0.5 \boxtimes (\llbracket \tilde{o}^{(h)} \rrbracket \boxplus \text{CONJ}(\llbracket \tilde{o}^{(h)} \rrbracket))$ 
      $\llbracket o^{(h+1)} \rrbracket \leftarrow -0.5i \boxtimes (\llbracket \tilde{o}^{(h)} \rrbracket \boxplus \text{CONJ}(\llbracket \tilde{o}^{(h)} \rrbracket))$ 
10:   $\llbracket \text{Att}^{(h)} \rrbracket \leftarrow \text{ATTENTION}(\llbracket Q^{(h)} \rrbracket, \llbracket K^{(h)} \rrbracket, \llbracket V^{(h)} \rrbracket)$ 
11:   $\llbracket \text{Att}^{(h+1)} \rrbracket \leftarrow \text{ATTENTION}(\llbracket Q^{(h+1)} \rrbracket, \llbracket K^{(h+1)} \rrbracket, \llbracket V^{(h+1)} \rrbracket)$ 
12:   $\llbracket \tilde{\text{Att}}^{(h)} \rrbracket \leftarrow \llbracket \text{Att}^{(h)} \rrbracket \boxplus i \boxtimes \llbracket \text{Att}^{(h+1)} \rrbracket$ 
13:   $\llbracket \tilde{Y} \rrbracket \leftarrow \llbracket \tilde{Y} \rrbracket \boxplus \llbracket \tilde{\text{Att}}^{(h)} \rrbracket \times \tilde{W}_0^{(h)}$ 
14: end for
15:  $\llbracket Y \rrbracket \leftarrow 0.5 \boxtimes (\llbracket \tilde{Y} \rrbracket \boxplus \text{CONJ}(\llbracket \tilde{Y} \rrbracket))$ 
16: return  $\llbracket Y \rrbracket$ 

```

For $\tilde{w}_{l,j}^{(h)} \in \tilde{W}_0^{(h)}$, the value of the j -th column of $\llbracket \tilde{\text{Att}}^{(h)} \rrbracket \times \tilde{W}_0^{(h)}$ is:

$$\llbracket \tilde{y}_j^{(h)} \rrbracket = \sum_{l=0}^{d'-1} \tilde{w}_{l,j}^{(h)} \cdot \llbracket \tilde{\text{att}}_l^{(h)} \rrbracket, j \in \{0, 1, \dots, d-1\} \quad (23)$$

Due to the conjugate structure of the weights, the real part of each term in the summation corresponds to the combined contribution of the two heads in that pair:

$$\begin{aligned} \text{Re}(\tilde{\text{Att}}^{(h)} \times \tilde{W}_0^{(h)}) &= \text{Att}^{(h)} W_0^{(h)} + \text{Att}^{(h+1)} W_0^{(h+1)} \\ &= [\mathbf{y}_0^{(h)} + \mathbf{y}_0^{(h+1)}, \dots, \mathbf{y}_{d-1}^{(h)} + \mathbf{y}_{d-1}^{(h+1)}] \end{aligned} \quad (24)$$

$\text{Re}(\cdot)$ denotes the real part. Consequently, the real part of the accumulated ciphertext $\llbracket \tilde{Y} \rrbracket$ yields the correct multi-head attention output Y . We perform a final real-part extraction:

$$\llbracket Y \rrbracket = 0.5 \boxtimes (\llbracket \tilde{Y} \rrbracket \boxplus \text{CONJ}(\llbracket \tilde{Y} \rrbracket)) = [\llbracket y_0 \rrbracket, \llbracket y_1 \rrbracket, \dots, \llbracket y_{d-1} \rrbracket] \quad (25)$$

This step recovers the real multi-head attention output Y , restoring the column-packed format for the next transformer block.

4.3 Adaptive Multi-Column Packing via Interleaved Batching

The column packing of MOAI and Euston achieves rotation-free PCMMs and non-linear operations, but it suffers from severe slot underutilization when the column length $L = t \cdot m$ is less than the slots S . This waste of capacity leads to constrained parallel efficiency and substantial ciphertext numbers.

To overcome these limitations, we propose AMCP, which consolidates data from multiple attention heads into a single ciphertext to enable multi-head parallel processing. By adopting an interleaved batching strategy [19, 25, 26], AMCP achieves simultaneous parallelism across both different batches and heads within a single ciphertext, while effectively facilitating the internal rotations.

Table 4: Selection of packing factor k under different batch sizes t (Assuming $S = 2^{15}$, $m = 128$, $H = 12$). The chosen k also represents the theoretical acceleration ratio of the attention layer compared to MOAI and Euston.

Batch size (t)	128	64	32
Packing factor (k) / Speedup	2	4	6

The design challenge of AMCP lies in structuring the input projection to achieve multi-head data packing and the output projection for multi-head data aggregation. This section provides a detailed design for both projections. Figure 4 illustrates examples of AMCP for the input and output projection.

Parallel Multi-Head Packing Strategy. We apply a *grouped packing* strategy to the input $X \in \mathbb{R}^{tm \times d}$. Define k as the packing factor, which is the number of feature columns per grouped ciphertext. d columns of the input matrix X are partitioned into $G = d/k$ groups, indexed by $g \in \{0, \dots, G-1\}$. To guarantee structural consistency, where the grouped ciphertext strictly mirrors the uniform multi-head output layout, k is derived as: $k = \max\{\kappa \in \text{Divisors}(H) \mid \kappa \cdot L \leq S\}$. $\text{Divisors}(H)$ is the set of factors of the attention head count H .

For BERT-base ($H = 12$), this constraint ($\kappa \in \{2, 3, 4, 6, 12\}$) ensures that the total number of ciphertext groups $G = d/k$ is a multiple of the head dimension d' . For instance, with $S = 2^{15}$, $m = 128$, $t = 32$, we derive $k = 6$, which compresses the query matrix Q from 768 to 128 ciphertexts. Since $2d' = 128$, the layout is perfectly aligned: the first 64 ciphertexts aggregate data from the first 6 heads, while the subsequent 64 ciphertexts cover the remaining 6 heads. This uniform interleaving is essential for unified parallel processing.

Interleaved Batching. Consider a grouped ciphertext with k feature columns and batch size t , forming $W = k \cdot t$ data streams $\{v^{(0)}, \dots, v^{(W-1)}\}$. These streams are logically ordered: indices 0 to $t-1$ cover all batches of the first feature column, followed by indices t to $2t-1$ for the second.

We construct the ciphertext $\llbracket C_g \rrbracket$ by interleaving the elements of W streams. Let $v_s^{(\zeta)}$ denote the element at sequence position s of the ζ -th stream. Specifically, we arrange elements based on their sequence index $s \in [0, m-1]$, inserting zero-padding after the active streams to align the total width to the stride $\phi = S/m$:

$$\llbracket C_g \rrbracket = [\underbrace{v_0^{(0)}, \dots, v_0^{(W-1)}}_{\text{Stride } \phi \text{ (Position 0)}}, \underbrace{0, \dots, v_1^{(0)}, \dots, v_1^{(W-1)}}_{\text{Stride } \phi \text{ (Position 1)}}, \dots] \quad (26)$$

This structure implies that:

- ① Adjacent slots in the ciphertext store data from consecutive batches or adjacent feature columns (vary ζ , fix s).
- ② Data from the same stream is distributed with a fixed stride ϕ , ensuring that the physical distance between sequence elements remains constant.

For example, as shown in Figure 4, the first grouped ciphertext with two feature columns x_0, x_1 forms data streams: $v^{(0)} = \{x_{00}, x_{10}\}, v^{(1)} = \{x'_{00}, x'_{10}\}, v^{(2)} = \{x_{01}, x_{11}\}, v^{(3)} = \{x'_{01}, x'_{11}\}$. The sequence of ciphertext $\llbracket C_g \rrbracket$ is $x_{00}, x'_{00}, x_{01}, x'_{01}, x_{10}, x'_{10}, x_{11}, x'_{11}$.

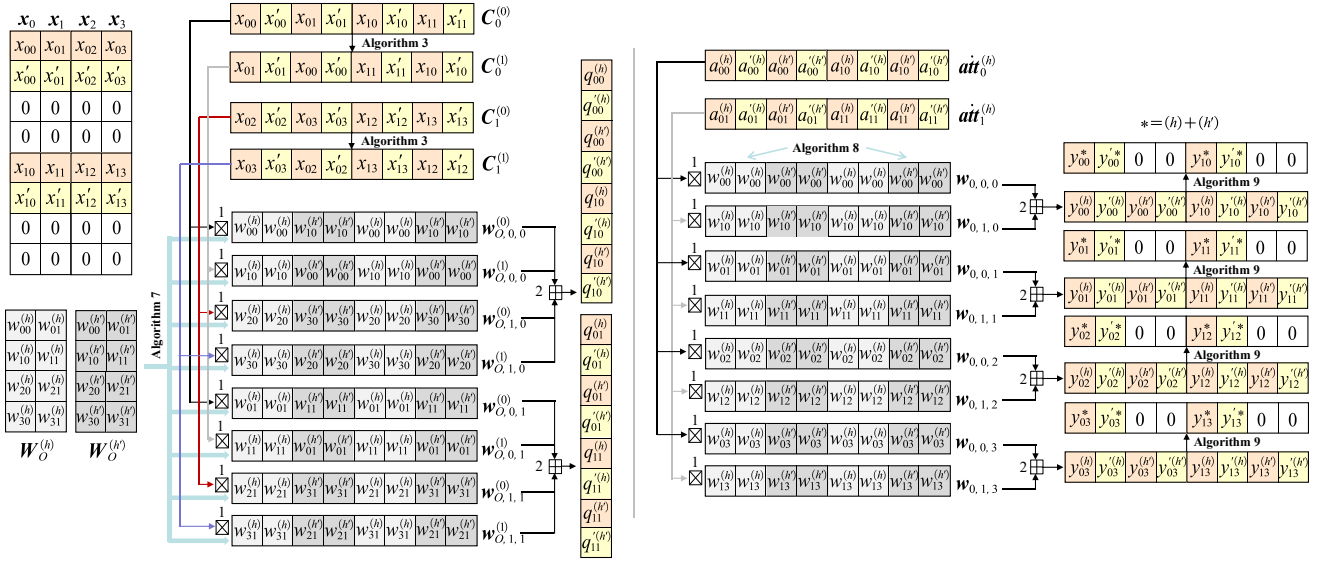


Figure 4: Examples of AMCP. The left side corresponds to the input projection, and the right side corresponds to the output projection. We set dimensions $m = d' = 2$, $d = 4$, batch size $t = 2$, slots $S = 8$, head number $H = 2$, and packing factor $k = 2$.

Algorithm 3 Iterative Group-Wise Rotation

- 1: **Input:** Ciphertext $\llbracket C_g \rrbracket$, batch size t , packing factor k , active width $W = k \cdot t$, total slots S , sequence length m .
- 2: **Output:** Aligned ciphertexts $\llbracket C_g^{(0)} \rrbracket, \dots, \llbracket C_g^{(k-1)} \rrbracket$.
- 3: $\varphi \leftarrow S/m$
- 4: $\llbracket C_g^{(0)} \rrbracket \leftarrow \llbracket C_g \rrbracket$
- 5: $u \leftarrow \underbrace{(1, \dots, 1, 0, \dots, 0)}_{W-t} \underbrace{, \dots, 0}_{t} \underbrace{, \dots, 0}_{\varphi-W}$
- 6: $u \leftarrow (u, u, \dots, u)$
- 7: $v \leftarrow \underbrace{(0, \dots, 0, 1, \dots, 1)}_{W-t} \underbrace{, \dots, 0}_{t} \underbrace{, \dots, 0}_{\varphi-W}$
- 8: $v \leftarrow (v, v, \dots, v)$
- 9: **for** $r = 0, 1, \dots, k-2$ **do**
- 10: $\llbracket C_g^{(r+1)} \rrbracket \leftarrow (\text{RotL}_t(\llbracket C_g^{(r)} \rrbracket) \boxtimes u) \boxplus (\text{RotR}_{W-t}(\llbracket C_g^{(r)} \rrbracket) \boxtimes v)$
- 11: **end for**
- 12: **return** $\llbracket C_g^{(0)} \rrbracket, \llbracket C_g^{(1)} \rrbracket, \dots, \llbracket C_g^{(k-1)} \rrbracket$

Interleaved Input Projection. The input projection aims to compute Q, K, V by summing the products of input columns and their corresponding weights. However, since the ciphertext $\llbracket C_g \rrbracket$ interleaves k distinct feature columns within a stride of φ , we must align these input columns to their target weights via rotation.

We employ an *Iterative Group-Wise Rotation* strategy to generate a set of k aligned states, denoted as $\{\llbracket C_g^{(r)} \rrbracket\}_{r=0}^{k-1}$. Starting from the initial state $\llbracket C_g^{(0)} \rrbracket = \llbracket C_g \rrbracket$, we iteratively derive each subsequent state $\llbracket C_g^{(r+1)} \rrbracket$ by cyclically shifting the active streams within the width W . This process is formally described in Algorithm 3.

Input Weight Construction and Aggregation. A detailed construction is provided in Appendix E.

Rotation Step Scaling for CCMMs. When applying AMCP to CCMMs, rotation indices must be scaled to account for the stride $\varphi = S/m$. Since logically adjacent elements are separated by φ slots in the packed ciphertext, any logical rotation of step ϕ required for standard operations maps to a scaling rotation of $\phi \cdot \varphi$.

Interleaved Output Projection. After the attention mechanism, the output attention $\llbracket \hat{A}_{att}^{(h)} \rrbracket = [\llbracket \hat{a}_{att}^{(h)} \rrbracket, \llbracket \hat{a}_{att}^{(h)} \rrbracket, \dots, \llbracket \hat{a}_{att}^{(h)} \rrbracket]$ retains the structure of the AMCP. Each ciphertext $\llbracket \hat{a}_{att}^{(h)} \rrbracket$ packs the attention results of k distinct heads. Unlike standard column packing, this distribution requires a specialized weight construction and a rotation-based accumulation to fuse the dispersed attentions.

Output Weight Construction and Aggregation. A detailed construction is provided in Appendix E.

4.4 Integration of RIHP, DHP, and AMCP

Appendix F presents the workflow for combining the first RIHP with DHP, and for integrating RIHP, DHP, and AMCP.

Appendix G theoretically compares the per-layer key-switching operations of STIP against Euston at batch size $t = 32$, and presents an amortized comparison against NEXUS, THOR, Powerformer, Euston, and MOAI at $t = 256$.

5 EFFICIENT NONLINEAR LAYERS

5.1 Gaussian Kernel-based Attention

To ensure the numerical stability and accelerate the attention mechanism, we employ the GK instead of Softmax. This reformulation eliminates the need for maximum value search and replaces the dynamic inverse with a constant.

Formulation. The attention score $S(Q, K)$ is computed by Eq. (2). Since the euclidean distance is non-negative, this formulation restricts outputs to the $(0, 1]$ range, eliminating the need for maximum value required by Softmax to prevent overflow. The normalization

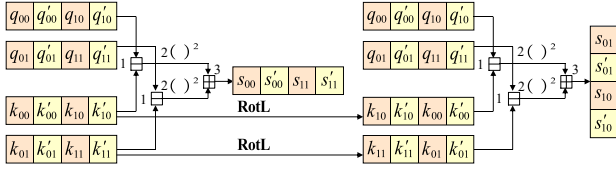


Figure 5: Examples of L2-norm under column packing. We set dimensions $m = d = d' = 2$ and batch size $t = 2$. The numbered operations are executed sequentially.

factor becomes a constant scalar (dependent only on d), replacing the dynamic inverse operation with a simple multiplication.

Efficient L2-Norm under Column Packing. The prior work [35] adopts a row-packing layout to compute the GK, obtaining L2-norm via intra-row rotation and summation. While efficient for row packing, applying this calculation method directly to column packing is computationally prohibitive.

We first address the structural incompatibility of row-based expansion by proposing a direct squared difference accumulation. As shown in Figure 5, this computes the r -th diagonal of $S(Q, K)$ for $r \in [0, m - 1]$:

$$[P_r] = \sum_{j=0}^{d'-1} ([q_j] \boxplus \text{RotL}_r([k_j]))^2, \quad (27)$$

where q_j and k_j denote the j -th feature columns of the query and key matrices. The number of rotations matches the attention score computed by MOAI.

To further minimize cost, we introduce the second RIHP to L2-norm computation with the squared expansion in Eq. (27): $\sum_{j=0}^{d'-1} ([q_j]^2 \boxplus (\text{RotL}_r([k_j]))^2 \boxplus 2 \boxtimes [q_j] \boxtimes \text{RotL}_r([k_j]))$. Algorithm 4 provides the second RIHP strategy to efficiently compute L2-norm, reducing the key-switch numbers from $(m - 1)d'$ to $(m + 1)d'/2 + 2(m - 1)$. Appendix H provides the pseudocode for the GK function and exponential function.

Depth Analysis. Compared to the 10 levels consumed by Softmax in MOAI and Euston, our GK mechanism requires merely 5 levels (1 for the PCMM and 4 for the exponential computation).

5.2 Inverse-Free Layer Normalization

LN typically requires computing $y = \gamma \cdot (x - \mu) / \sigma + \beta$, which involves calculating the inverse of the standard deviation $(1/\sigma)$. This operation is expensive in CKKS. To overcome this bottleneck, we propose a scale propagation strategy, eliminating the need for inverse computation.

Inverse Elimination via Scale Invariance. To bypass the inverse square root, we reformulate the first LN to propagate the scaling factor. Currently, Euston implements LN with the minimal multiplicative level. The standard LN in Euston is defined as:

$$y_i = \gamma \cdot \frac{x_i - \mu}{\sigma} + \beta = \gamma \cdot \frac{z_i \sqrt{n\lambda}}{\sqrt{(\lambda \sum_{i=1}^n z_i^2 - 1) + 1}} + \beta \quad (28)$$

where $z_i = x_i - \mu = \frac{1}{n} \sum_{j=1}^n (x_i - x_j)$, and λ constrains $\sum_{i=1}^n z_i^2$ within $[0, 2]$ for square root approximations. However, computing (28) is expensive, requiring 9 multiplicative levels.

Algorithm 4 L2-Norm Computation with the Second RIHP

```

1: Input: Query  $[Q] = \{[q_j]\}_{j=0}^{d'-1}$ , key  $[K] = \{[k_j]\}_{j=0}^{d'-1}$ .
2: Output: Encrypted diagonals  $\{[P_r]\}_{r=0}^{m-1}$ .
3:  $[S_Q] \leftarrow 0, [S_K] \leftarrow 0$ 
4: for  $j = 0, \dots, d'/2 - 1$  do
5:    $[\hat{q}_j] \leftarrow [q_j] \boxplus i \cdot [q_{j+d'/2}]$ 
6:    $[\hat{k}_j] \leftarrow [k_j] \boxplus i \cdot [k_{j+d'/2}]$ 
7:    $[\hat{k}'_j] \leftarrow \text{CONJ}([\hat{k}_j])$ 
8:    $[S_Q] \leftarrow [S_Q] \boxplus [\hat{q}_j] \boxtimes \text{CONJ}([\hat{q}_j])$ 
9:    $[S_K] \leftarrow [S_K] \boxplus [\hat{k}_j] \boxtimes [\hat{k}'_j]$ 
10: end for
11: for  $r = 0, \dots, m - 1$  do
12:    $[C_r] \leftarrow \sum_{j=0}^{d'/2-1} [\hat{q}_j] \boxtimes \text{RotL}_r([\hat{k}'_j])$ 
13:    $[T_r] \leftarrow [S_Q] \boxplus \text{RotL}_r([S_K]) \boxplus ([C_r] \boxplus [C_r])$ 
14:    $[P_r] \leftarrow ([T_r] \boxplus \text{CONJ}([T_r])) \boxtimes 0.5$ 
15: end for
16: return  $\{[P_r]\}_{r=0}^{m-1}$ 

```

Our strategy multiplies the entire equation by σ , yielding a scaled output $\hat{y}_i$:

$$\hat{y}_i = \gamma z_i \sqrt{n\lambda} + \beta \sqrt{(\lambda \sum_{i=1}^n z_i^2 - 1) + 1} = \gamma z_i \sqrt{n\lambda} + \beta \sigma \quad (29)$$

This effectively converts the inverse into a multiplication, treating σ as a scaling factor to the output data.

Integration with Subsequent Layers. The scaled output $\hat{y}$ is then processed by the subsequent linear layer. To maintain mathematical consistency, the scaling factor σ must be preserved. While the weight multiplication $W\hat{y}$ naturally scales σ , the bias term b must be adjusted to match this scale. We achieve this by multiplying the bias by σ :

$$\hat{z} = W\hat{y} + \sigma \cdot b = \sigma(Wy + b) \quad (30)$$

Here, $\hat{z}$ represents the scaled input for the second LN. Crucially, this scaling factor naturally cancels out in the subsequent standard LN due to its scale-invariance property. By substituting the scaled input $\hat{z}_i = \sigma z_i$ into Eq. (28), the expansion factor σ appears in both the numerator and the denominator:

$$\frac{\hat{z}_i \sqrt{n\lambda}}{\sqrt{(\lambda \sum_{i=1}^n \hat{z}_i^2 - 1) + 1}} = \frac{\sigma z_i \sqrt{n\lambda}}{\sqrt{\sigma^2 \lambda \sum_{i=1}^n z_i^2 - 1 + 1}} = \frac{z_i \sqrt{n\lambda}}{\sqrt{\lambda \sum_{i=1}^n z_i^2}} \quad (31)$$

This confirms that the expansion factor σ is eliminated by the division. Thus, the proposed method completely removes the inverse operation from the first LN without affecting the correctness.

Appendix H presents the pseudocode for computing this inverse-free LN and the square root evaluation.

Scale-Preserving Activation Function. To support this inverse-free formulation, we ensure that the subsequent non-linear activation functions satisfy the homogeneity property (i.e., $f(\alpha x) = \alpha f(x)$ for $\alpha > 0$). Since GELU does not strictly adhere to this, we adopt ReLU, which guarantees that the scaling factor propagate correctly through the feed-forward network. Notably, as shown in Table 5 and 10, replacing GELU with ReLU (including the substitution of Softmax with GK) preserves the model's accuracy [36].

Depth Analysis. By eliminating the inverse approximation, the depth requirement of the first LN drops from 9 levels to 5 levels (1 for mean calculation, 2 for the square root, 2 for two PCMMs).

6 PERFORMANCE EVALUATION

6.1 Experimental Setup

Building upon the NEXUS¹ and Euston² architecture, we engineered the STIP using C++, designed to extend model support across both GPU and CPU. Similar to NEXUS and Euston, STIP implements the RNS-CKKS scheme using Phantom [37] for the GPU and Microsoft SEAL [38] for the CPU. In our experiments, we utilize BERT-base for micro-benchmarks on both GPU and CPU. For end-to-end evaluations, we employ BERT-base, LLAMA-3-8B, and GPT-2-1.5B, conducted specifically on GPU.

Parameter Configuration: In the end-to-end inference, we configured the polynomial degree to $N = 2^{16}$ and utilized a 1763-bit composite modulus, which ensures a security level of 128 bits [39]. For a fair comparison, we adopted the identical bootstrapping algorithm used in Euston, which supports 14 multiplicative levels before bootstrapping is required. In the micro-benchmarks, we configured $N = 2^{16}$. The composite modulus and the modulus chain length are configured according to the specific micro-benchmark settings.

GPU Environment: GPU acceleration was evaluated using an NVIDIA A100 featuring 80 GB of VRAM, operating within the CUDA 12.2 runtime environment.

CPU Environment: Experiments were executed on an AMD EPYC 7542 processor (2.90 GHz). The system ran Ubuntu 22.04.1 LTS. The thread count was standardized at 32.

6.2 Micro-benchmarks

We evaluated our proposed techniques against NEXUS and Euston via GPU and CPU micro-benchmarks on the BERT-base model, as illustrated in Figure 6.

CCMM for Attention Scores (QK^T). This benchmark evaluates the execution time of CCMM for QK^T . We configured STIP with a higher modulus chain length ($L = 5$) to support the imaginary unit operation and conjugate unpacking, compared to MOAI ($L = 3$) and Euston. As shown in Figure 6 (a), STIP significantly outperforms the baselines. Across varying token lengths, STIP reduces execution time by 45.1% (GPU) and 33.1% (CPU) compared to MOAI. Against Euston, the efficiency gains are even more substantial, reducing runtime by over 85% on GPU and 78% on CPU. Furthermore, we evaluated the computational cost of the L2-norm using the second RIHP strategy, where its execution time is nearly identical to the QK^T computation in STIP.

Throughput of Linear Projections. We assess the throughput of linear projections (XW_O , $O \in \{Q, K, V\}$) using DHP and AMCP strategies separately. First, under standard settings ($m = 128$, $t = 256$), STIP with DHP consistently outperforms baselines with a higher modulus chain $L = 4$ ($L = 3$ for baselines) for conjugate unpacking. As shown in Figure 6 (b), it reduces total execution time

Table 5: Inference accuracy evaluation on GLUE benchmark.

Model	Dataset	Softmax + GELU			GK + ReLU	
		Plaintext	NEXUS	Euston	Plaintext	STIP
BERT-base	RTE	70.76%	70.60%	70.76%	72.20%	72.06%
	SST-2	92.66%	92.41%	92.55%	92.73%	92.62%
	QNLI	91.67%	91.27%	91.52%	91.21%	91.06%
LLAMA-3-8B	RTE	82.75%	81.24%	81.56%	82.46%	81.32%
	SST-2	94.94%	94.46%	94.67%	94.39%	94.01%
	QNLI	90.70%	90.20%	90.38%	90.36%	90.12%

by approximately 46.1%/57.7% (GPU) and 33.4%/57.2% (CPU) compared to Euston/NEXUS. Second, under small batchsize ($t = 64$), we configured STIP with a higher modulus chain length of $L = 6$ ($L = 3$ for baselines) to accommodate the additional operations in Algorithm 3. As shown in Figure 6 (c), AMCP maximizes slot utilization, reducing execution time by 48.2%/59.4% (GPU) and 50.1%/67.9% (CPU) compared to Euston/NEXUS. Specially, AMCP extends to all attention computations. The cumulative performance benefits of its parallelism are demonstrated in our end-to-end experiments.

Storage Overhead. We evaluate the memory footprint for storing ciphertexts (Q, K, V) with $m = 128$ and $t = 64$. Leveraging the AMCP strategy, STIP packs four heads per ciphertext, achieving SOTA memory efficiency comparable to BOLT⁴. As shown in Figure 6 (d), for a 12-head configuration, STIP requires only 576 MB, a 75% reduction compared to the 2,304 MB needed by Euston.

Non-linear Layers Efficiency. We assess the efficiency of non-linear operations using our GK mechanism and inverse-free LN. For non-linear activation, STIP reduces the modulus chain to $L = 7$ ($L = 12/18$ for Euston/NEXUS), yielding a 42.4%/85.2% (GPU) and 77.4%/99.7% (CPU) time reduction. Notably, AMCP allows STIP to maintain constant low latency (3.84 ms) even at small batch sizes (2^6), eliminating the latency spikes observed in Euston. For LN, our inverse-free approach reduces the depth to $L = 6$ ($L = 11/20$ for Euston/NEXUS). Under saturation ($t \geq 2^8$), STIP achieves a 30%/86.8% reduction in amortized execution time on GPU and up to 69.6%/99% on CPU compared to Euston/NEXUS.

6.3 Accuracy

To evaluate inference accuracy, we assessed STIP on the RTE, SST-2, and QNLI datasets from the GLUE benchmark [40]. We employed the BERT-base and LLAMA-3-8B fine-tuned for each task and compared the performance of STIP against the plaintext baseline, Euston, and NEXUS. As shown in Table 5, STIP exhibits acceptable deviations from the plaintext baseline and achieves performance comparable to both Euston and NEXUS. We also evaluated the inference accuracy of GPT-2-1.5B in Table 10 of Appendix I.

6.4 End-to-end Inference Evaluation

Comparisons with Euston. Consistent with the evaluation methodologies in Euston⁵, we analyze the runtime breakdown of STIP and Euston on the BERT-base, LLAMA-3-8B, and GPT-2-1.5B.

¹<https://github.com/zju-abclab/NEXUS>

²<https://github.com/FLL-Lab/Euston>

³Since MOAI represents the SOTA for CCMMs, we evaluate the MOAI's CCMMs. Moreover, NEXUS does not release codes for CCMMs.

⁴We evaluate BOLT, which adopts a compact packing method.

⁵Euston evaluates a single transformer layer under batch size $t = 32$ with setting the modulus chain to the absolute depth consumed.

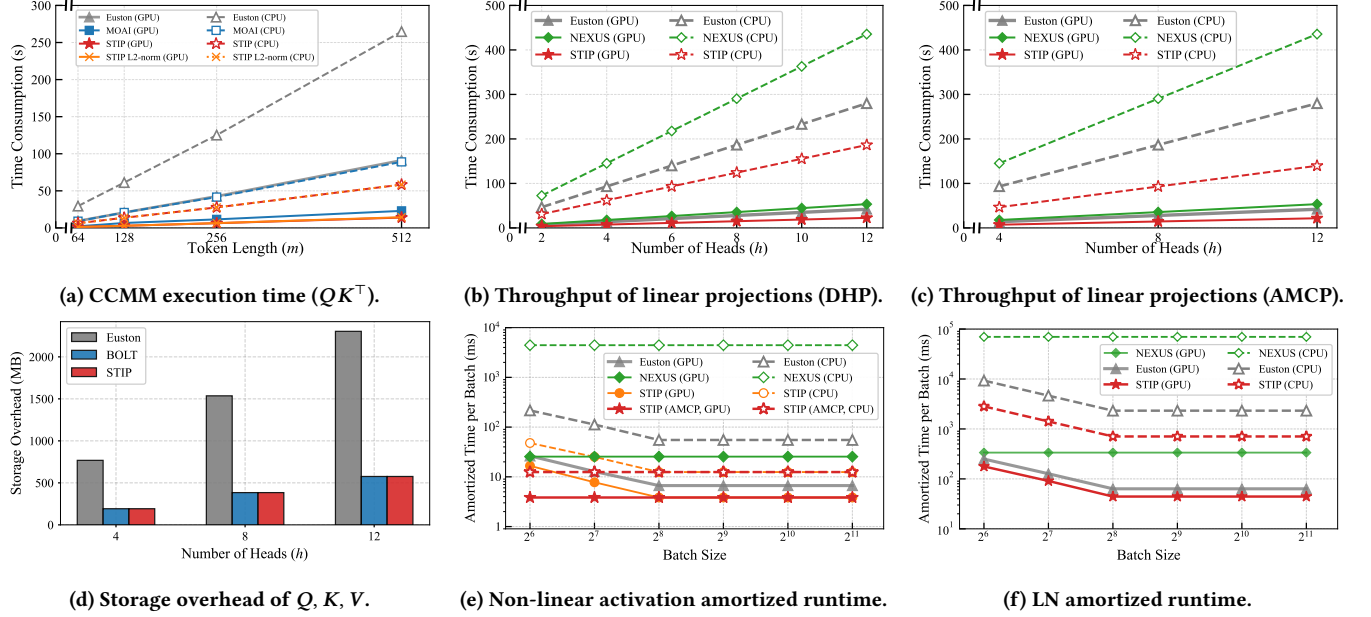


Figure 6: Micro-benchmark results in the GPU and CPU environments

Table 6: Breakdown of execution time on an A100 GPU compared to Euston under BERT-base.

STIP			Euston		
Operation	Depth	Time (s)	Operation	Depth	Time (s)
Input projection	17→10	0.15	Input projection	17→16	1.31
Attention score	10→8	0.18	Attention score	16→14	7.74
GK	8→3	0.07	Softmax	14→4	0.64
Attention output	3→1	0.04	Attention output	4→2	7.54
Bootstrappings	1→8	2.38	Output projection	2→1	0.48
Output projection	8→6	0.1	Bootstrappings	1→10	15.73
LN1	6→1	0.36	LN1	10→1	0.5
Bootstrappings	1→5	13.11	Bootstrappings	1→15	18.51
FC1	5→4	1.74	FC1	15→14	1.73
Activation layer	4→2	3.53	Activation layer	14→2	7.28
FC2	2→1	1.75	FC2	2→1	1.74
Bootstrappings	1→10	15.73	Bootstrappings	1→10	15.73
LN2	10→1	0.5	LN2	10→1	0.5
Bootstrappings	1→17	20.17	Bootstrappings	1→17	20.17
Classification	-	3.5	Classification	-	3.5
Total	-	63.31	Total	-	103.1

Following the experimental setting in Euston, both frameworks are configured with $t = 32$ and $m = 128$. As detailed in Table 6 and 11, STIP demonstrates a significant advantage in end-to-end inference, achieving approximately 1.6×6 speedups over Euston on BERT-base, LLAMA-3-8B, and GPT-2-1.5B. The results of Table 11 are presented in the Appendix I.

⁶The $1.6 \times$ speedup is a conservative estimate. The actual gain is higher since Euston’s methodology—measuring via absolute modulus chain consumption—overlooks the fact that many of its operations are performed at higher levels.

Runtime breakdown analysis: The performance disparity is primarily driven by optimizations in the attention layer, non-linear layer, and bootstrapping:

① Attention Layer: STIP exhibits lower latency across the attention layer, reducing the total execution time from Euston’s 33.44s to 2.92s, achieving an $11.5 \times$ speedup. This overall reduction in execution time is driven by our AMCP, which enables a single ciphertext to process data from multiple attention heads in parallel. Building on this foundation, the DHP strategy accelerates the throughput of input and output projections, while the RIHP strategy speeds up computations of the attention score and attention output.

② Nonlinear Layer: By replacing Softmax with the GK and leveraging AMCP for parallel execution, STIP reduces the operation time from 0.64s to 0.07s ($9.1 \times$ speedup). Additionally, optimizations in LN1 eliminate costly inverse operations, reducing the runtime from 0.5s to 0.36s ($1.4 \times$ speedup).

③ Bootstrapping Optimizations: STIP achieves a total bootstrapping time of 51.39s, representing reductions of $1.36 \times$ compared to Euston (70.14s). This efficiency is driven by the first bootstrapping, which processes only 128 ciphertexts via AMCP compared to Euston’s 768, yielding a $6.6 \times$ speedup on BERT-base. Furthermore, the lower multiplicative depth consumption of the optimized LN1 and ReLU allows subsequent bootstrapping operations to restore ciphertexts to a lower level.

Comparisons with MOAI, THOR, and Powerformer. Consistent with the evaluation methodologies in MOAI, we analyze the runtime breakdown of STIP, MOAI, THOR, and Powerformer on the BERT-base. As detailed in Table 7⁷, STIP achieves approximately $2.3 \times$, $4.9 \times$, and $2.8 \times$ speedups over MOAI, THOR, and Powerformer

⁷Our implementation utilizes the same Phantom library and identical measurement methodology as MOAI. Accordingly, the performance metrics for MOAI, THOR, and Powerformer are drawn from the MOAI paper, all evaluated on an A100 GPU.

Table 7: Breakdown of execution time on an A100 GPU compared to MOAI, THOR, and Powerformer under BERT-base. “†” indicates $t = 256$. “‡” indicates $t = 128$.

Operation	STIP [†] /STIP [‡]	MOAI [†] /MOAI [‡]	THOR	Powerformer
Input projection	7.2/3.32	16.54/33.08	49.77	57.65
Attention score	6.5/5.21	14.07/28.14	16.25	28.76
GK / Softmax	0.78/0.66	2.19/4.38	15.53	0.75
Attention output	0.2/0.16	0.29/0.58	13.08	18.95
Output projection	0.25/0.65	0.48/0.96	27.43	22.54
LN1	0.54/1.08	0.15/0.3	7.13	0.37
FC1	7.87/15.74	16.18/32.36	49.8	59.21
Activation layer	5.05/10.13	3.3/6.6	29.42	8.31
FC2	2.88/5.76	2.88/5.76	49.19	43.77
LN2	0.86/1.5	0.15/0.3	4.1	0.3
Bootstrappings	91.1/160	227.84/455.68	337.86	103.72
Classification	0.7/1.2	0.7/1.2	2.7	0.2
Total	123.92/207.48	283.95/569.34	602.26	344.52

at batch size $t = 256$. At $t = 128$, STIP’s speedup over MOAI further increases to 2.7 $\times$.

Performance at $t = 256$: The speedup over MOAI is primarily driven by RIHP, DHP, and the reduced multiplicative depth of non-linear layers. Although STIP’s non-linear operations (LN1, LN2, and activation layer) take slightly longer, their lower depth consumption significantly accelerates bootstrapping, which is the main computational bottleneck in FHE.

Performance at $t = 128$: With the AMCP strategy, the speedup advantage over MOAI expands. STIP still outperforms MOAI and Powerformer. Unlike MOAI, whose runtime severely degrades at this smaller batch size (far exceeding Powerformer and approaching THOR), AMCP effectively restrains STIP’s latency growth. Compared to STIP at $t = 256$, AMCP accelerates input projections by reducing the PCMM count. Furthermore, since the first bootstrapping at $t = 128$ is scheduled before the output projection (instead of after in $t = 256$), the attention layer operates at lower multiplicative levels, resulting in faster overall computations.

7 RELATED WORK

7.1 Packing Method and Matrix Multiplication

Single-Sequence Packing Methods. Schemes like THOR and Powerformer operate in a single-input style. They often pack data from the transformer’s multiple heads into a single ciphertext to leverage SIMD parallelism. While this approach achieves much lower absolute latency for an isolated request, its runtime typically increases linearly with the number of sequence tokens. Furthermore, these methods suffer from spatial misalignment, necessitating rotation-intensive PCMMs and non-linear aggregation. Specifically, BOLT faces high latency due to suboptimal complexity, while THOR’s diagonal strategy incurs key-switching overheads for non-square projections. About the complex domain, early work [41] utilized the imaginary part for tensors. While Powerformer also utilizes the complex domain, our optimization goals differ fundamentally. For optimized CCMMs, based on [42], Powerformer uses complex conjugates to aggregate independent matrix products, which leaves the rotation complexity unchanged. In contrast, our

RIHP provides an intrinsic algorithmic optimization, fundamentally reducing the rotation complexity for QK^T from md' to $md'/2$. Regarding optimized PCMMs, Powerformer reduces rotations in compact packing by pairing vectors, yet the rotational overhead remains. Conversely, our DHP exploits the complex domain within a rotation-free column-wise strategy, processing two heads simultaneously to double throughput.

Batch Packing Methods. Schemes like NEXUS, MOAI, and Euston support batch inference, processing multiple sequences in a single execution. Although this batch style incurs a longer wall-clock time, it utilizes larger batch sizes to amortize the cost of expensive homomorphic operations, significantly reducing the average per-sequence latency. Moreover, MOAI and Euston achieve rotation-free PCMMs and non-linear layers by packing each feature column to distinct ciphertexts. However, this incurs severe slot underutilization, low throughput and excessive ciphertext numbers. Furthermore, computing CCMMs still necessitates substantial homomorphic rotations, dominating the total rotation overhead.

7.2 Optimization of Non-linear Layers

Softmax Optimizations. Current strategies face three limitations. Fixed-Maximum Simplification (NEXUS, MOAI, Euston) uses a pre-determined constant to avoid dynamic maximum search but introduces severe numerical instability (overflow/underflow) and difficulty in evaluating row maximums in sparse layouts. Iterative Precision (THOR) maintains precision but relies on computationally intensive iterative algorithms for inverse approximation, resulting in excessive multiplicative depth and key-switching operations. Polynomial Replacement (Powerformer) replaces Softmax with BP-Max via knowledge distillation, treating non-linearity as a black box and limiting theoretical robustness.

LN Optimizations. Optimization efforts generally fall into two categories. Direct Homomorphic Approximation (NEXUS, THOR, MOAI, Euston) frameworks evaluate the standard LN function, which is bottlenecked by the high multiplicative depth required for the inverse square root. Distillation-based Replacement (Powerformer) replaces dynamic statistics with learned constants via knowledge distillation. It necessitates a data-dependent re-training process, limiting its generality for various data and models.

8 CONCLUSION

In this paper, we presented STIP, a comprehensive framework designed to overcome the critical computational and storage bottlenecks in secure non-interactive transformer inference. By leveraging RIHP and DHP, we effectively halve the rotation overhead in CCMMs and the execution time of input and output projections. Concurrently, our AMCP strategy resolves severe slot wastage to maximize parallel throughput while significantly reducing the ciphertext footprint. We replace Softmax with GK to ensure efficiency and stability by eliminating the need for both maximum value search and inverse operation. Additionally, we reformulate the LN into an inverse-free form to reduce the multiplicative depth and expensive key-switch operations. Experimental evaluations on an A100 GPU validate the superiority of our approach, demonstrating that STIP achieves 1.6 $\times$, 2.3 $\times$, 4.9 $\times$, and 2.8 $\times$ speedups over Euston, MOAI, THOR, and Powerformer, respectively.

REFERENCES

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the Advances in Neural Information Processing Systems*. 5998–6008.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*. 4171–4186.
- [3] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.
- [4] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothee Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- [5] Congcong Chen, Lifei Wei, Jintao Xie, and Yang Shi. 2025. Privacy-preserving machine learning based on cryptography: a survey. *ACM Transactions on Knowledge Discovery from Data*, 19(4):1–33.
- [6] R. Lehmkuhl, P. Mishra, A. Srinivasan, and R. A. Popa. 2021. Muse: Secure inference resilient to malicious clients. In *USENIX Security Symposium*. 2201–2218.
- [7] Bosen Rao, Jiale Zhang, Di Wu, Chengcheng Zhu, Xiaobing Sun, and Bing Chen. 2025. Privacy inference attack and defense in centralized and federated learning: a comprehensive survey. *IEEE Transactions on Artificial Intelligence*, 6(2):333–353.
- [8] Jabari Kwesi, Jiaxun Cao, Riya Manchanda, and Pardis Emami-Naeini. 2025. Exploring user security and privacy attitudes and concerns toward the use of general-purpose LLM chatbots for mental health. In *USENIX Security Symposium*. 6007–6024.
- [9] M. Nasr, N. Carlini, J. Hayase, M. Jagielski, A. F. Cooper, D. Ippolito, C. A. Choquette-Choo, E. Wallace, F. Tram'èr, and K. Lee. 2023. Scalable extraction of training data from (production) language models. *arXiv preprint arXiv:2311.17035*.
- [10] Yuntian Chen, Tianpei Lu, Zhanyong Tang, Bingsheng Zhang, Zhiying Shi, Yuxiang Luan, and Zhuzhu Wang. 2026. SoK: Private Transformer-Based Model Inference. *Cryptology ePrint Archive, Paper 2026/491*.
- [11] Qi Pang, Jinhao Zhu, Helen Mo llering, Wenting Zheng, and Thomas Schneider. 2024. Bolt: Privacy-preserving, accurate and efficient inference for transformers. In *IEEE Symposium on Security and Privacy (SP)*. 4753–4771.
- [12] Kanav Gupta, Neha Jawalkar, Ananta Mukherjee, Nishanth Chandran, Divya Gupta, Ashish Panwar, and Rahul Sharma. 2023. Sigma: Secure gpt inference with function secret sharing. *Cryptology ePrint Archive, Paper 2023/1269*.
- [13] Wenjie Lu, Zhicong Huang, Zhen Gu, Jingyu Li, Jian Liu, Kui Ren, Cheng Hong, Tao Wei, and WenGuang Chen. 2025. Bumblebee: Secure two-party inference framework for large transformers. In *Network and Distributed System Security (NDSS)*.
- [14] Tianshi Xu, Wen-jie Lu, Jiangrui Yu, Yi Chen, Chenqi Lin, Runsheng Wang, and Meng Li. 2025. Breaking the layer barrier: Remodeling private transformer inference with hybrid CKKS and MPC. In *USENIX Security Symposium*. 2653–2672.
- [15] Andes Y. L. Kei and Sherman S. M. Chow. 2025. SHAFT: Secure, handy, accurate, and fast transformer inference. In *Network and Distributed System Security (NDSS)*.
- [16] Jiawen Zhang, Jian Liu, Xinpeng Yang, Yinghao Wang, Kejia Chen, Xiaoyang Hou, Kui Ren, and Xiaohu Yang. 2025. Secure transformer inference made noninteractive. In *Network and Distributed System Security Symposium (NDSS)*.
- [17] Dongjin Park, Eunsang Lee, and Joon-Woo Lee. 2025. Powerformer: Efficient privacy-preserving transformer with batch rectifier-power max function and optimized homomorphic attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*. 11090–11111.
- [18] Jungho Moon, Dongwoo Yoo, Xiaoqian Jiang, and Miran Kim. 2025. THOR: Secure transformer inference with homomorphic encryption. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*.
- [19] Linru Zhang, Xiangning Wang, Jun Jie Sim, Zhicong Huang, Jiahao Zhong, Huaxiong Wang, Pu Duan, and Kwok-Yan Lam. 2026. MOAI: Module-optimizing architecture for non-interactive secure transformer inference. In *International Conference on Learning Representations (ICLR)*.
- [20] Xinwen Gao, Shaojing Fu, Lin Liu, Zhuotao Liu, Yuchuan Luo, and Yongjun Wang. 2026. Euston: Efficient and user-friendly secure transformer inference with non-interactivity. In *IEEE Symposium on Security and Privacy (SP)*.
- [21] Wenxuan Zeng, Tianshi Xu, Yi Chen, Yifan Zhou, Mingzhe Zhang, Jin Tan, Cheng Hong, and Meng Li. 2025. Towards efficient privacy-preserving machine learning: A systematic review from protocol, model, and system perspectives. *arXiv preprint arXiv:2507.14519*.
- [22] Alexander Viand, Patrick Jattke, Miro Haller, and Anwar Hithnawi. 2023. HECO: Fully homomorphic encryption compiler. In *USENIX Security Symposium*. 4715–4732.
- [23] Ahmad Al Badawi, Bharadwaj Veeravalli, Jie Lin, Nan Xiao, Matsumura Kazuaki, and Aung Khin Mi Mi. 2020. Multi-gpu design and performance evaluation of homomorphic encryption on gpu clusters. *IEEE Transactions on Parallel and Distributed Systems*, 32(2):379–391.
- [24] Microsoft batch-inference. 2023. <https://github.com/microsoft/batch-inference>, January 2023. Microsoft Research, Redmond, WA.
- [25] Allon Adir, Ehud Aharoni, Nir Drucker, Ronen Levy, Hayim Shaul, and Omri Soceanu. 2024. Homomorphic Encryption for Data Science (HE4DS). *Springer*.
- [26] Ehud Aharoni, Allon Adir, Moran Baruch, Nir Drucker, Gilad Ezov, Ariel Farkash, Lev Greenberg, Ramy Masalha, Guy Moshkovich, Dov Murik, Hayim Shaul, and Omri Soceanu. 2023. Helayers: A tile tensors framework for large neural networks on encrypted data. In *Proceedings on Privacy Enhancing Technologies*.
- [27] Shai Halevi and Victor Shoup. 2021. Bootstrapping for helib. *Journal of Cryptology*, 34(1):7.
- [28] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
- [29] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology—ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security*. 409–437.
- [30] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2019. A full rms variant of approximate homomorphic encryption. In *Selected Areas in Cryptography—SAC 2018: 25th International Conference*. 347–368.
- [31] Jiachen Lu, Jinghan Yao, Junge Zhang, Xiatian Zhu, Hang Xu, Weiguo Gao, Chunjing Xu, Tao Xiang, and Li Zhang. 2021. SOFT: Softmax-free transformer with linear complexity. In *Advances in Neural Information Processing Systems*. 21297–21309.
- [32] Yifan Chen, Qi Zeng, Heng Ji, and Yun Yang. 2021. Skyformer: Remodel self-attention with gaussian kernel and nystrom method. In *Advances in Neural Information Processing Systems*. 2122–2135.
- [33] Eunsang Lee, Joon-Woo Lee, Jong-Seon No, and Young-Sik Kim. 2021. Minimax approximation of sign function by composite polynomial for homomorphic comparison. *IEEE Transactions on Dependable and Secure Computing*, 19(6):3711–3727.
- [34] Jung Hee Cheon, Dongwoo Kim, and Duhyeon Kim. 2020. Efficient homomorphic comparison methods with optimal complexity. In *Advances in Cryptology—ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security*. 221–256.
- [35] Donghwan Rho, Taeseong Kim, Minje Park, Jung Woo Kim, Hyunsik Chae, Jung Hee Cheon, and Ernest K Ryu. 2025. Encryption-friendly llm architecture. In *International Conference on Learning Representations (ICLR)*.
- [36] Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. 2024. ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models. In *International Conference on Learning Representations (ICLR)*.
- [37] Hao Yang, Shiyu Shen, Wangchen Dai, Lu Zhou, Zhe Liu and Yunlei Zhao. 2024. Phantom: A cuda-accelerated word-wise homomorphic encryption library. *IEEE Transactions on Dependable and Secure Computing*, 21(5):4895–4906.
- [38] Microsoft SEAL (release 4.1). 2023. <https://github.com/Microsoft/SEAL>, January 2023. Microsoft Research, Redmond, WA.
- [39] Martin Albrecht, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Shai Halevi, Jeffrey Hoffstein, Satya Lokam, and Daniele Micciancio. 2021. Homomorphic encryption standard. In *Protecting Privacy through Homomorphic Encryption*. 31–62.
- [40] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. 2018. Glue: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*. 353–355.
- [41] Ehud Aharoni, Nir Drucker, Gilad Ezov, Hayim Shaul, and Omri Soceanu. 2022. Complex encoded tile tensors: Accelerating encrypted analytics. *IEEE Security & Privacy*, 20(5):35–43.
- [42] Xiaoqian Jiang, Miran Kim, Kristin Lauter, and Yongsoo Song. 2018. Secure outsourced matrix computation and application to neural networks. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 1209–1222.
- [43] Wonhee Cho, Guillaume Hanrot, Taeseong Kim, Minje Park, and Damien Stehlé. 2024. Fast and accurate homomorphic softmax evaluation. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 4391–4404.
- [44] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. 2016. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*. 1525–1534.
- [45] Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, Leo Gao, Jonathan Tow, Baber Abbasi, Alham Fikri Aji, et al. 2024. Lessons from the Trenches on Reproducible Evaluation of Language Models. *arXiv preprint arXiv:2405.14782*.

A OPEN SCIENCE

We have taken comprehensive steps to ensure the transparency, reproducibility, and extensibility of this work. To support independent evaluation of our core contributions, we release an anonymized artifact package containing all materials required for replication: (1) the full source code for both GPU and CPU implementations (including the STIP, Euston, and NEXUS modules), with build files (CMakeLists.txt) and third-party cryptographic dependencies (PhantomFHE for GPU and modified SEAL for CPU); (2) data-generation scripts for model inputs (Data/generate_.py); (3) training and evaluation code for GK attention experiments (pretrain/pretrain_gaussian.py, eval_sst2.py, eval_rte.py, eval_qnli.py), together with an experiment orchestration script (run_all_experiments.sh); (4) environment specifications (pretrain/requirements.txt) and platform requirements (CUDA/cuSOLVER, compiler/toolchain versions); (5) usage documentation at repository root and module level (Readme.md, GPU/Readme.md, CPU/Readme.md, pretrain/README.md), including commands to reproduce preprocessing, pretraining, and downstream benchmarks; and (6) benchmark outputs and logs, including key metrics such as runtime and numerical error.

All datasets and models used in this study are publicly available and properly cited in the manuscript. The program committee and reviewers can access the complete artifact via an anonymous channel: <https://anonymous.4open.science/r/STIP>. The released package includes detailed experiment logs and all necessary scripts to reproduce the reported results. We hope these resources will facilitate rigorous verification and enable the community to build upon our work, fostering further innovation in privacy-preserving inference.

B ETHICAL CONSIDERATIONS

We discuss the ethical considerations of our research on FHE for privacy-preserving inference. Our work is guided by the below principles: (1) Beneficence. This work aims to optimize the performance of privacy-preserving inference. We hope that our research will raise awareness of data security and encourage the deployment of practical and efficient privacy computing systems. (2) Respect for Persons. This work utilizes exclusively publicly available datasets and open-source models. It respects the intellectual property of developers and ensures individual privacy by enabling secure inference without processing any sensitive or personally identifiable information. (3) Justice. The results of this work will be shared with the broader research community, including developers and researchers. This ensures that the benefits of efficient secure computation are widely disseminated to promote equitable access to privacy technologies. (4) Respect for Law and Public Interest. We are committed to respecting all relevant laws and the public interest. Our research complies with legal frameworks and ensures that efficiency-driven optimizations do not introduce new security vulnerabilities.

C THE SECOND RIHP STRATEGY

Algorithm 5 provides the pseudocode of the second RIHP strategy to efficiently compute the attention score QK^T . Different from Algorithm 1, we pack pairs of feature columns into the real and imaginary components of a single complex ciphertext, such that

Algorithm 5 The Second RIHP Strategy

```

1: Input: Query  $\llbracket Q \rrbracket = \{\llbracket q_j \rrbracket\}_{j=0}^{d'-1}$ , key  $\llbracket K \rrbracket = \{\llbracket k_j \rrbracket\}_{j=0}^{d'-1}$ .
2: Output: Encrypted diagonals of  $QK^T$ :  $\{\llbracket P_r \rrbracket\}_{r=0}^{m-1}$ .
3: for  $j = 0, \dots, d'/2 - 1$  do
4:    $\llbracket \hat{q}_j \rrbracket \leftarrow \llbracket q_j \rrbracket \boxplus i \cdot \llbracket q_{j+d'/2} \rrbracket$ 
5:    $\llbracket \hat{k}_j \rrbracket \leftarrow \llbracket k_j \rrbracket \boxplus i \cdot \llbracket k_{j+d'/2} \rrbracket$ 
6: end for
7: for  $r = 0, \dots, m-1$  do
8:    $\llbracket V_r \rrbracket \leftarrow \sum_{j=0}^{d'/2-1} \llbracket \hat{q}_j \rrbracket \boxtimes \text{RotL}_r(\llbracket \hat{k}_j \rrbracket)$ 
9:    $\llbracket P_r \rrbracket \leftarrow (\llbracket V_r \rrbracket \boxplus \text{CONJ}(\llbracket V_r \rrbracket)) \boxtimes 0.5$ 
10: end for
11: return  $\{\llbracket P_r \rrbracket\}_{r=0}^{m-1}$ 

```

Algorithm 6 Attention Output with the Second RIHP Strategy

```

1: Input: Activation output  $\llbracket S \rrbracket = \{\llbracket s_k \rrbracket\}_{k=0}^{m-1}$ , value matrix  $\llbracket V \rrbracket = \{\llbracket v_k \rrbracket\}_{k=0}^{d'-1}$ .
2: Output: Encrypted columns of  $SV$ :  $\{\llbracket att_j \rrbracket\}_{j=0}^{d'-1}$ .
3:  $b \leftarrow \lceil \sqrt{m} \rceil$ ,  $g \leftarrow \lceil m/b \rceil$ 
4: for  $k = 0, \dots, d'/2 - 1$  do
5:    $\llbracket \hat{v}_k \rrbracket \leftarrow \llbracket v_k \rrbracket \boxplus i \cdot \llbracket v_{k+d'/2} \rrbracket$ 
6: end for
7: for  $k = 0, \dots, d'/2 - 1$  do
8:   for  $r = 0, \dots, b-1$  do
9:      $\beta_r \leftarrow \text{RotL}_r(\llbracket \hat{v}_k \rrbracket)$ 
10:   end for
11:    $\llbracket \hat{att}_k \rrbracket \leftarrow \sum_{\alpha=0}^{g-1} \text{RotL}_{\alpha b}(\sum_{r=0}^{b-1} \text{RotL}_{m-\alpha b}(\llbracket s_{ab+r} \rrbracket) \boxtimes \beta_r)$ 
12:    $\llbracket att_k \rrbracket \leftarrow (\llbracket \hat{att}_k \rrbracket \boxplus \text{CONJ}(\llbracket \hat{att}_k \rrbracket)) \boxtimes 0.5$ 
13:    $\llbracket att_{k+d'/2} \rrbracket \leftarrow (\llbracket \hat{att}_k \rrbracket \boxminus \text{CONJ}(\llbracket \hat{att}_k \rrbracket)) \boxtimes (-0.5i)$ 
14: end for
15: return  $\{\llbracket att_j \rrbracket\}_{j=0}^{d'-1}$ 

```

$\llbracket \hat{q}_j \rrbracket \leftarrow \llbracket q_j \rrbracket \boxplus i \cdot \llbracket q_{j+d'/2} \rrbracket$ and $\llbracket \hat{k}_j \rrbracket \leftarrow \llbracket k_j \rrbracket \boxplus i \cdot \llbracket k_{j+d'/2} \rrbracket$. Leveraging the algebraic identity $\text{Re}(z \cdot \bar{w}) = \text{Re}(z)\text{Re}(w) + \text{Im}(z)\text{Im}(w)$, the conjugate multiplication $\llbracket \hat{q}_j \rrbracket \boxtimes \text{RotL}_r(\llbracket \hat{k}_j \rrbracket)$ naturally aggregates the dot product contributions from both feature column pairs (i.e., $q_j k_j + q_{j+d'/2} k_{j+d'/2}$) into the real component, while isolating the cross-terms in the imaginary part. Crucially, this method halves the total number of key switches from $(m-1)d'$ to $(m-1)d'/2 + m$.

Algorithm 6 shows the Baby-Step Giant-Step (BSGS)-optimized CCMM for SV by incorporating the second RIHP strategy, reducing the total rotation number from $2\sqrt{md'} + m$ to $\sqrt{md'} + m + d'/2$.

CCMM for Attention Output (SV). We configured STIP with a higher modulus chain length of $L = 4$ to support the conjugate unpacking, compared to MOAI ($L = 3$) and Euston ($L = 4$). As shown in Figure 7, across varying token lengths, STIP reduces time by 32% compared to MOAI and 97.1% compared to Euston.

D THE CORRECTNESS OF THE FIRST RIHP STRATEGY

Theorem 1 (Correctness of the first RIHP strategy)

Let $Q, K \in \mathbb{R}^{m \times d'}$ be the query and key matrices, and let $P = QK^T \in \mathbb{R}^{m \times m}$ be the attention score matrix. Suppose m is even.

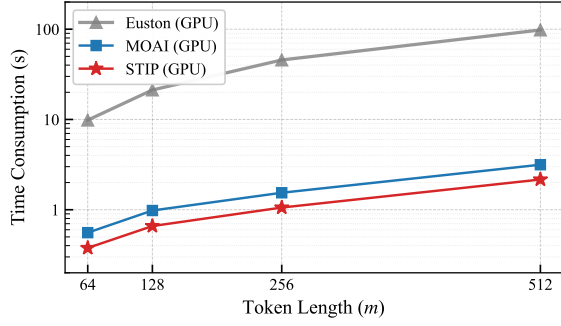


Figure 7: CCMM execution time (SV).

Let $\hat{K} \in \mathbb{C}^{m \times d'}$ be a hybrid key matrix where the l -th column is constructed as $\hat{k}_l = k_l + i \cdot \text{RotL}_{m/2}(k_l)$, for $l \in [d']$. For any rotation index $r \in [0, m/2-1]$, the hybrid ciphertext $\llbracket C_r \rrbracket$ computed by the operation:

$$\llbracket C_r \rrbracket = \sum_{l=0}^{d'-1} \llbracket q_l \rrbracket \boxtimes \text{RotL}_r(\llbracket \hat{k}_l \rrbracket) \quad (32)$$

encrypts the r -th diagonal of P in its real part and the $(r + m/2)$ -th diagonal of P in its imaginary part. That is:

$$\llbracket C_r \rrbracket = \llbracket P_r \rrbracket \boxplus i \cdot \llbracket P_{r+m/2} \rrbracket \quad (33)$$

Proof: According to the definition of $\llbracket C_r \rrbracket$, we expand $\hat{k}_l$:

$$\llbracket C_r \rrbracket = \sum_{l=0}^{d'-1} \llbracket q_l \rrbracket \boxtimes \text{RotL}_r(\llbracket k_l \rrbracket \boxplus i \cdot \text{RotL}_{m/2}(\llbracket k_l \rrbracket)) \quad (34)$$

Since the rotation operation $\text{RotL}_r(\cdot)$ is linear over the field of complex numbers (i.e., $\text{RotL}_r(A + B) = \text{RotL}_r(A) + \text{RotL}_r(B)$):

$$\llbracket C_r \rrbracket = \sum_{l=0}^{d'-1} \llbracket q_l \rrbracket \boxtimes (\text{RotL}_r(\llbracket k_l \rrbracket) \boxplus i \cdot \text{RotL}_r(\text{RotL}_{m/2}(\llbracket k_l \rrbracket))) \quad (35)$$

Due to $\text{RotL}_a(\text{RotL}_b(v)) = \text{RotL}_{a+b}(v)$, we simplify the imaginary term:

$$\llbracket C_r \rrbracket = \sum_{l=0}^{d'-1} (\llbracket q_l \rrbracket \boxtimes \text{RotL}_r(\llbracket k_l \rrbracket) \boxplus i \cdot \llbracket q_l \rrbracket \boxtimes \text{RotL}_{r+\frac{m}{2}}(\llbracket k_l \rrbracket)) \quad (36)$$

By the distributivity of summation, we can separate the real and imaginary components:

$$\llbracket C_r \rrbracket = \sum_{l=0}^{d'-1} \llbracket q_l \rrbracket \boxtimes \text{RotL}_r(\llbracket k_l \rrbracket) \boxplus i \cdot \sum_{l=0}^{d'-1} \llbracket q_l \rrbracket \boxtimes \text{RotL}_{r+\frac{m}{2}}(\llbracket k_l \rrbracket) \quad (37)$$

For $j \in [m]$, the j -th diagonal of QK^T is $\llbracket P_j \rrbracket = \sum_{l=0}^{d'-1} \llbracket q_l \rrbracket \boxtimes \text{RotL}_j(\llbracket k_l \rrbracket)$. Substituting this equation into Eq. (37): The Real Part corresponds to index $j = r$, equals $\llbracket P_r \rrbracket$. The Imaginary Part corresponds to index $j = r + m/2$, equals $\llbracket P_{r+m/2} \rrbracket$. Therefore:

$$\llbracket C_r \rrbracket = \llbracket P_r \rrbracket \boxplus i \cdot \llbracket P_{r+m/2} \rrbracket \quad (38)$$

Algorithm 7 Input Weight Packing Algorithm

```

1: Input: Batch  $t$ , sequence  $m$ , packing  $k$ , total slots  $N$ , weight
    $w_{i,j}^{(h)} \in \{W_O\}_{O \in \{Q,K,V\}, h \in \{0,k,2k,\dots,H-k\}}$ .
2: Output:  $\{w_{O,g,j}^{(r)}\}_{g \in \{0,\dots,G-1\}, j \in \{0,\dots,d'-1\}, r \in \{0,\dots,k-1\}}$ .
3:  $\varphi \leftarrow S/m$ 
4: for  $g = 0, 1, \dots, G-1$  do
5:   for  $j = 0, 1, \dots, d'-1$  do
6:      $w \leftarrow [\underbrace{w_{gk,j}^{(h)}, \dots, w_{gk+1,j}^{(h)}}_t, \dots, \underbrace{w_{gk+k-1,j}^{(h+k-1)}, \dots, 0}_{\varphi-k \cdot t}]$ 
7:      $w_{O,g,j}^{(0)} \leftarrow [\underbrace{w, w, \dots, w}_m]$ 
8:     for  $r = 1, 2, \dots, k-1$  do
9:        $\hat{w} \leftarrow [\underbrace{w_{gk+r \bmod k,j}^{(h)}, \dots, w_{gk+r+1 \bmod k,j}^{(h+1)}}_t, \dots, \underbrace{w_{gk+r \bmod k,j}^{(h+k-1)}, \dots, 0}_{\varphi-k \cdot t}]$ 
10:       $w_{O,g,j}^{(r)} \leftarrow [\underbrace{\hat{w}, \hat{w}, \dots, \hat{w}}_m]$ 
11:    end for
12:  end for
13: end for
14: return  $\{w_{O,g,j}^{(r)}\}_{g \in \{0,\dots,G-1\}, j \in \{0,\dots,d'-1\}, r \in \{0,\dots,k-1\}}$ 

```

Algorithm 8 Output Weight Packing Algorithm

```

1: Input: Weight  $w_{i,j}^{(h)} \in W_0^{(h)}$ ,  $h \in \{0, k, 2k, \dots, H-k\}$ , batch  $t$ ,
   sequence  $m$ , packing  $k$ , stride  $\varphi = S/m$ .
2: Output:  $\{w_{0,i,j}\}_{i \in \{0,\dots,d'-1\}, j \in \{0,\dots,d-1\}}$ 
3: for  $i = 0, 1, \dots, d'-1$  do
4:   for  $j = 0, 1, \dots, d-1$  do
5:      $w \leftarrow [\underbrace{w_{i,j}^{(h)}, \dots, w_{i,j}^{(h+1)}}_t, \dots, \underbrace{w_{i,j}^{(h+k-1)}, \dots, 0}_{\varphi-k \cdot t}]$ 
6:      $w_{0,i,j} \leftarrow [\underbrace{w, w, \dots, w}_m]$ 
7:   end for
8: end for
9: return  $\{w_{0,i,j}\}_{i \in \{0,\dots,d'-1\}, j \in \{0,\dots,d-1\}}$ 

```

E AMCP WEIGHT CONSTRUCTION

Input Weight Construction and Aggregation. To perform the input projection over packed ciphertexts, we compute the output

for the j -th dimension $\dot{o}_j (o \in \{q, k, v\}, j \in \{0, \dots, d' - 1\})$ as:

$$\llbracket \dot{o}_j^{(h)} \rrbracket = \sum_{g=0}^{G-1} \sum_{r=0}^{k-1} \llbracket C_g^{(r)} \rrbracket \boxtimes \mathbf{w}_{O,g,j}^{(r)} \quad (39)$$

The sequence of packed weight matrices $\mathbf{w}_{O,g,j}^{(r)} \in W_{O,g}$ acts as a dynamic selector, constructed via Algorithm 7. After aggregation, the input projection yields $\llbracket \dot{O}^{(h)} \rrbracket = [\llbracket \dot{o}_0^{(h)} \rrbracket, \dots, \llbracket \dot{o}_{d'-1}^{(h)} \rrbracket]$, $h \in \{0, k, 2k, \dots, H - k\}$, where each ciphertext $\llbracket \dot{o}_j^{(h)} \rrbracket$ packs the results of k distinct heads. Algorithm 7 operates on plaintext matrices and can be executed during the offline phase. Consequently, it incurs no computational overhead for online inference.

Output Weight Construction and Aggregation. To perform the output projection over packed attentions, we compute the output for the j -th dimension $\dot{y}_j^{(h)}$, ($j \in \{0, \dots, d - 1\}$) as:

$$\llbracket \dot{y}_j^{(h)} \rrbracket = \sum_{i=0}^{d'-1} \llbracket \text{ait}_i^{(h)} \rrbracket \boxtimes \mathbf{w}_{0,i,j} \quad (40)$$

The sequence of packed weight matrices $\mathbf{w}_{0,i,j} \in W_{0,i}$ constructed via Algorithm 8. After aggregation, the output linear projection yields $\llbracket Y^{(h)} \rrbracket = [\llbracket \dot{y}_0^{(h)} \rrbracket, \dots, \llbracket \dot{y}_{d-1}^{(h)} \rrbracket]$, $h \in \{0, k, 2k, \dots, H - k\}$. It is worth noting that Algorithm 8 operates on plaintext matrices, and its computational overhead is negligible.

However, the equation above is incomplete. In our AMCP, k distinct heads are distributed across different slots within the ciphertext $\llbracket \dot{y}_j^{(h)} \rrbracket$. Let $\llbracket \mathbf{y}_j^{(v)} \rrbracket$ denote the encrypted output projection result corresponding to the j -th feature dimension of the v -th attention head, where $v \in \{0, 1, 2, \dots, H - 1\}$. To complete the output projection and obtain the final result $\llbracket \mathbf{y}_j \rrbracket = \sum_{v=0}^{H-1} \llbracket \mathbf{y}_j^{(v)} \rrbracket$, we must sum the contributions from all these heads. As shown in Algorithm 9, we shift the results from different slots into alignment and aggregate them to reconstruct the full summation.

F INTEGRATED WORKFLOWS

Integrated Workflow of RIHP and DHP. As illustrated in Figure 8, the architecture effectively integrates the RIHP and DHP strategies to maximize computational density. The workflow is orchestrated in a "Split-Compute-Merge" paradigm, processing H attention heads in $H/2$ parallel groups.

Phase 1: DHP-based Input Projection. The process begins with the input X encoded in the standard column-packed format. We apply DHP to the input projection. By utilizing pre-coded complex weight matrices $\tilde{W}_Q^{(h)}, \tilde{W}_K^{(h)}, \tilde{W}_V^{(h)}$, we simultaneously compute the projections for two adjacent heads (head h and $h + 1$) within a single PCMM operation.

The hybrid ciphertexts $\tilde{Q}^{(h)}, \tilde{K}^{(h)}, \tilde{V}^{(h)}$ encode the data for head h in the real component and head $h + 1$ in the imaginary component.

Phase 2: Unpacking and Layout Transformation. Proceeding to the middle phase, we perform the conjugate unpacking to separate the complex ciphertexts back into independent real-valued ciphertexts for each head. Notably, as indicated by the $\hat{K}^{(h)}$ node, the key matrix undergoes a RIHP. This prepares the key matrix for the rotation-optimized attention calculation in the subsequent step.

Phase 3: RIHP-based CCMMs. First, we perform the CCMM between the query matrices $Q^{(h)}$ and the hybrid key matrices $\hat{K}^{(h)}$. For $r \in [0, m/2 - 1]$, we compute the hybrid diagonal ciphertext $C^{(h)} = \{C_r^{(h)}\}$. Crucially, due to the RIHP, $C_r^{(h)}$ simultaneously

Algorithm 9 Output Aggregation Algorithm

```

1: Input: Ciphertexts  $\{\llbracket \dot{y}_j^{(h)} \rrbracket \mid h \in \{0, k, \dots, H - k\}\}$ , batch size  $t$ , packing factor  $k$ .
2: Output: Aggregated result  $\llbracket \mathbf{y}_j \rrbracket$ .
3:  $\llbracket S \rrbracket \leftarrow 0, \varphi \leftarrow S/m$ 
4: for  $h = 0, k, 2k, \dots, H - k$  do
5:    $\llbracket Y \rrbracket \leftarrow \llbracket \dot{y}_j^{(h)} \rrbracket$ 
6:   if  $k \in \{2^n\}_{n \geq 1}$  then
7:     for  $i = 0, 1, \dots, \log_2 k - 1$  do
8:        $\delta \leftarrow 2^i \cdot t$ 
9:        $\llbracket Y \rrbracket \leftarrow \llbracket Y \rrbracket \boxplus \text{RotL}_\delta(\llbracket Y \rrbracket)$ 
10:    end for
11:   else
12:      $\llbracket \omega \rrbracket \leftarrow \llbracket Y \rrbracket$ 
13:     for  $i = 1, 2, \dots, k - 1$  do
14:        $\delta \leftarrow i \cdot t$ 
15:        $\llbracket Y \rrbracket \leftarrow \llbracket Y \rrbracket \boxplus \text{RotL}_\delta(\llbracket \omega \rrbracket)$ 
16:     end for
17:   end if
18:    $\llbracket S \rrbracket \leftarrow \llbracket S \rrbracket \boxplus \llbracket Y \rrbracket$ 
19: end for
20:  $s \leftarrow (\underbrace{1, \dots, 1}_t, \underbrace{0, \dots, 0}_{\varphi-t})$ 
21:  $\hat{s} \leftarrow (\underbrace{s, s, \dots, s}_m)$ 
22:  $\llbracket \mathbf{y}_j \rrbracket \leftarrow \llbracket S \rrbracket \boxtimes \hat{s}$ 
23: return  $\llbracket \mathbf{y}_j \rrbracket$ 

```

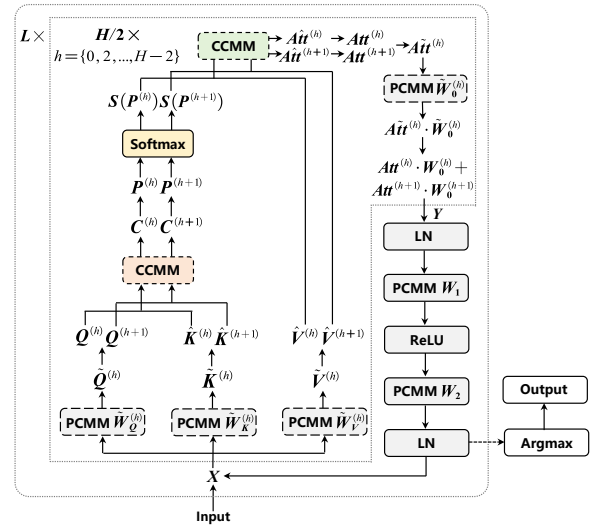


Figure 8: The integrated workflow of RIHP and DHP.

encodes the r -th diagonal of $P^{(h)}$ in its real part and the $(r + m/2)$ -th diagonal in its imaginary part. Second, to recover the standard diagonal packing format, we apply the conjugate unpacking to each $C_r^{(h)}$. The recovered ciphertexts constitute the score matrix

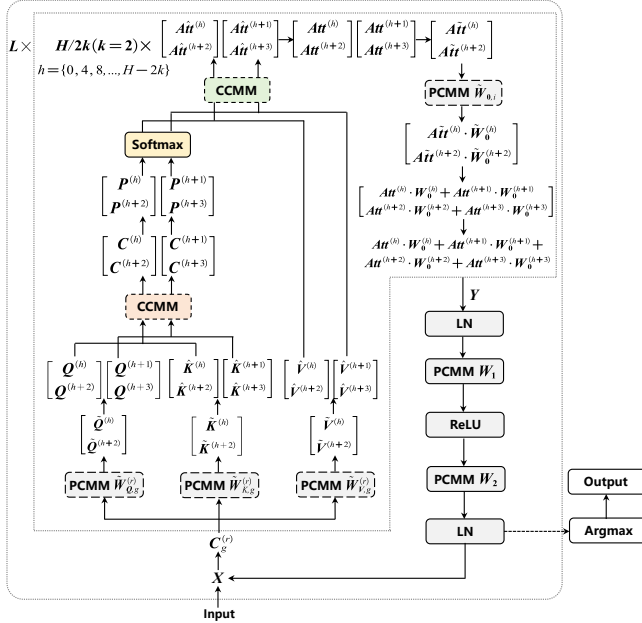


Figure 9: The integrated workflow of RIHP, DHP, and AMCP.

$P^{(h)}$, which is then processed by the Softmax to obtain $S^{(h)}$. Subsequently, we execute the CCMM between $S^{(h)}$ and the hybrid value matrices $\hat{V}^{(h)}$ to produce the attention output $\hat{Att}^{(h)}$. The conjugate unpacking then retrieves the attention output $Att^{(h)}$.

Phase 4: DHP-based Output Projection. In the final output projection phase, we re-introduce DHP to aggregate the results. The independent outputs are repacked into a complex ciphertext $\hat{Att}^{(h)}$. We then perform the output projection using the pre-coded conjugate weight matrix $\tilde{W}_0^{(h)}$. The property of this complex multiplication naturally accumulates the contributions from both heads into the real component. Finally, the aggregated result Y is extracted.

Integrated Workflow of RIHP, DHP, and AMCP. As illustrated in Figure 9, the architecture effectively integrates the RIHP, DHP, and AMCP. For ease of exposition, we consider the case ($k = 2$) and assume a block-wise packing format, which in practice corresponds to interleaved packing.

For the projection, we first merge weights from adjacent heads through DHP, followed by the permutation of the weight matrix via AMCP. Upon completion of the input projection, a dual-head conjugate unpacking is performed, followed by the rotation operation to generate the vector $\hat{K}^{(h)}, \hat{V}^{(h)}$. Subsequently, two CCMMs are performed, and the outputs $P^{(h)}, Att^{(h)}$ are obtained through the conjugate unpacking of $C^{(h)}, \hat{Att}^{(h)}$. For the output projection, the DHP is applied, followed by the unpacking of the output. Finally, the projected results from disparate heads are aggregated.

G PERFORMANCE COMPARISONS

Performance Comparison with Euston. Table 8 presents a comparison of the key-switching operations between Euston and STIP under batch size $t = 32$. By integrating RIHP, DHP, and AMCP strategies, STIP achieves a remarkable reduction in complexity. The

Table 8: The key-switching operations required for one transformer layer under batch size $t = 32$: (1) Input projection XW_0 ; (2) Attention scores computation QK^T ; (3) Activation (Softmax/GK); (4) Activation output S multiplied by V ; (5) Output projection XW_0 ; (6) LN and FFN; Concrete parameters are based on BERT-base model: $m = 128, d = 768, d' = 64, H = 12, N = 2^{16}, t = 32, k = 6$ is the packing factor in STIP. $G = d/k = 128$ is the number of grouped ciphertexts in STIP.

ID	Scheme	Key Switch Equation	Number
(1)	Euston	0	0
	STIP	$2(k-1)G$	1280
(2)	Euston	$2(m-1)d'H$	195072
	STIP	$((d'+1)m/2)(H/k)$	8320
(3)	Euston	0	0
	STIP	0	0
(4)	Euston	$2(m-1)d'H$	195072
	STIP	$(\sqrt{md'} + m + d'/2)(H/k)$	1768
(5)	Euston	0	0
	STIP	$d(k-1)$	3840
(6)	Euston	0	0
	STIP	0	0
Total	Euston	-	390144
	STIP	-	15208

total number of key switches required for one transformer layer decreases from 390144 in Euston to just 15208 in STIP, representing a 96.1% reduction. Specifically, the total key switches required for QK^T/SV decrease by 96.8%/99.1%. This is driven by the effect of AMCP (which parallelizes rotations across heads) and RIHP (which decreases the number of rotations). Although STIP introduces a minor overhead in the linear projections totaling 5120 rotations due to Algorithm 3 and Algorithm 9, this cost is negligible compared to the massive savings obtained in other components.

Performance Comparison with NEXUS, THOR, Powerformer, Euston, and MOAI. Table 9 presents the amortized key-switching operations under batch size $t = 256$. STIP achieves the lowest total overhead, reducing rotations by 98.8%, 91.9%, 83.2%, 84.4%, and 47.9% compared to NEXUS, THOR, Powerformer, Euston, and MOAI, respectively. We analyze the breakdown as follows:

PCMMs and Non-linear Layers: NEXUS and THOR suffer from severe rotation overheads in PCMMs and non-linear layers due to spatial misalignment and iterative evaluations. While Powerformer achieves zero rotations in the non-linear layer, it heavily relies on complex, data-dependent pre-training and knowledge distillation to eliminate accumulation steps. In contrast, STIP, Euston, and MOAI adopt column-wise batch packing, structurally eliminating rotations in PCMMs and non-linear Layers.

CCMMs Bottleneck: With rotations eliminated elsewhere, CCMMs (QK^T and SV) become the primary bottleneck for Euston (1524 rotations) and MOAI (455 rotations). STIP breaks this bottleneck via RIHP. This halves the CCMM rotation overhead compared to MOAI (dropping to 195 for QK^T and 42 for SV).

Table 9: Amortized number of key-switching operations for one transformer layer under batch size $t = 256$: (1) Input projection XW_O ; (2) Attention scores computation QK^T ; (3) Activation layer; (4) Activation output S multiplied by V ; (5) Output projection XW_O ; (6) LN and FFN. The concrete parameters are as follows: $m = 128$, $d = 768$, $d' = 64$, $b = 3072$, $H = 12$, $N = 2^{16}$, $a = 64$, and $c = 16$.

ID	Scheme	Key switch	
		Equation	Number
(1)	NEXUS	$2md/(N/2m)$	768
	THOR	$\frac{m}{2} + 3(H-2)\frac{d'}{c} + \frac{3d'}{c}$	192
	Powerformer	$\approx \sqrt{\frac{5}{9}} \cdot 2\sqrt{\frac{md(3d)}{N/2}}$	122
	Euston/MOAI	0	0
	STIP	0	0
(2)	NEXUS	$m(d'+1)H/(N/4m)$	780
	THOR	$\frac{m}{2c}(\frac{m}{2}+7) + \frac{m}{8}(\log c+3) + \frac{m}{2}$	460
	Powerformer	$\approx (17+14\sqrt{2})\sqrt{a}+3a$	486
	Euston	$(m-1)d'H/(N/4m)$	762
	MOAI	$(m-1)d'H/(N/2m)$	381
	STIP	$((d'+1)m/2)H/(N/2m)$	195
(3)	NEXUS/THOR	$\lceil \log_2(m) \rceil \frac{m}{\frac{N}{4}2^{\lceil \log_2(m) \rceil}} \cdot H$	84
	Powerformer/ Euston/MOAI	0	0
	STIP	0	0
(4)	NEXUS	N/A	N/A
	THOR	$m(\log c+3)/4 + m\frac{m+6}{4c}$	492
	Powerformer	$\approx (19+10\sqrt{2})\sqrt{a}+3a$	457
	Euston	$(m-1)d'H/(N/4m)$	762
	MOAI	$(2\sqrt{md'}+m)H/(N/2m)$	74
	STIP	$(\sqrt{md'}+m+d'/2)H/(N/2m)$	42
(5)	NEXUS	$2d^2/(N/4m)$	9216
	THOR	$\frac{d'}{2} + 2(\frac{H}{2}-1)\frac{m}{c}$	122
	Powerformer	$\approx \sqrt{\frac{2}{3}} \cdot 2\sqrt{\frac{md^2}{N/2}}$	75
	Euston/MOAI	0	0
	STIP	0	0
(6)	NEXUS	$\frac{2m(d+b)}{N/4m} + \lceil \log_2(d) \rceil \frac{2d}{\frac{N}{4}2^{\lceil \log_2(d) \rceil}}$	8640
	THOR	$m + \frac{2(b+d)}{c} + \lceil \log_2(d) \rceil \frac{2d}{\frac{N}{4}2^{\lceil \log_2(d) \rceil}}$	1568
	Powerformer	$\approx 2\sqrt{2} \cdot \sqrt{\frac{mdb}{N/2}}$	272
	Euston/MOAI	0	0
	STIP	0	0
Total	NEXUS	-	> 19488
	THOR	-	2918
	Powerformer	-	1412
	Euston	-	1524
	MOAI	-	455
	STIP	-	237

H NON-LINEAR ALGORITHMS

This appendix presents the algorithmic implementations for non-linear functions. Algorithm 10 provides the computations for the nonlinear layers, including the GK, LN, and ReLU.

The homomorphic exponential approximation [43] (Algorithm 11) adopts the scaling decomposition, where the input is scaled down by a factor of 2^τ to ensure it falls within the bounded interval

Algorithm 10 Non-linear Functions in STIP

```

1: Input: Column or diagonal-packed input  $\llbracket X \rrbracket = \{\llbracket x_i \rrbracket\}_{i=1}^m$ .
2: Output: Column or diagonal-packed output  $\llbracket Y \rrbracket = \{\llbracket y_i \rrbracket\}_{i=1}^m$ .
3: Function: Gaussian Kernel
4: for  $i = 1, 2, \dots, m$  do
5:    $\llbracket y_i \rrbracket \leftarrow \text{EXP}\left(-\frac{1}{2\sqrt{d}} \boxtimes \llbracket x_i \rrbracket\right)$ 
6: end for
7: Return  $\llbracket Y \rrbracket = [\llbracket y_1 \rrbracket, \llbracket y_2 \rrbracket, \dots, \llbracket y_m \rrbracket]$ 

8: Function: Inverse-Free LN
9: for  $i = 1, 2, \dots, m$  do
10:   for  $j = 1, 2, \dots, m$  do
11:      $\llbracket a_j \rrbracket \leftarrow \llbracket x_i \rrbracket \boxtimes \llbracket x_j \rrbracket$ 
12:   end for
13:    $\llbracket z_i \rrbracket \leftarrow \frac{1}{m} \boxtimes \sum_{j=1}^m \llbracket a_j \rrbracket$ 
14: end for
15:  $\llbracket v \rrbracket \leftarrow \text{SQRT}(\lambda \boxtimes \sum_{i=1}^m (\llbracket z_i \rrbracket \boxtimes \llbracket z_i \rrbracket))$ 
16: for  $i = 1, 2, \dots, m$  do
17:    $\llbracket y_i \rrbracket \leftarrow (\gamma\sqrt{m\lambda} \boxtimes \llbracket z_i \rrbracket) \boxplus (\beta \boxtimes \llbracket v \rrbracket)$ 
18: end for
19: Return  $\llbracket Y \rrbracket = [\llbracket y_1 \rrbracket, \llbracket y_2 \rrbracket, \dots, \llbracket y_m \rrbracket], \llbracket v \rrbracket$ 

20: Function: ReLU
21: for  $i = 1, 2, \dots, m$  do
22:    $\llbracket s_i \rrbracket \leftarrow \text{SGN}(\llbracket x_i \rrbracket)$ 
23:    $\llbracket y_i \rrbracket \leftarrow \llbracket x_i \rrbracket \boxtimes (\llbracket s_i \rrbracket \boxplus 1) \boxtimes 0.5$ 
24: end for
25: Return  $\llbracket Y \rrbracket = [\llbracket y_1 \rrbracket, \llbracket y_2 \rrbracket, \dots, \llbracket y_m \rrbracket]$ 

```

Algorithm 11 Homomorphic Exponential Approximation

```

1: Input: Ciphertext  $\llbracket x \rrbracket$ , polynomial  $f(x) = \sum_{i=0}^7 a_i x^i$ , positive integer  $\tau \leq 3$ .
2: Output:  $\llbracket y \rrbracket \approx \text{EXP}(\llbracket x \rrbracket)$ .
3:  $\llbracket z \rrbracket \leftarrow \llbracket x \rrbracket \boxtimes \frac{1}{2^\tau}$ 
4:  $\llbracket y \rrbracket \leftarrow f(\llbracket z \rrbracket)$ 
5: for  $i = 1, 2, \dots, \tau$  do
6:    $\llbracket y \rrbracket \leftarrow \llbracket y \rrbracket \boxtimes \llbracket y \rrbracket$ 
7: end for
8: Return  $\llbracket y \rrbracket$ 

```

Algorithm 12 Homomorphic Square Root Approximation

```

1: Input: Ciphertext  $\llbracket x \rrbracket$  (Assuming  $x \in (0, 2)$ ).
2: Output:  $\llbracket y \rrbracket \approx \text{SQRT}(\llbracket x \rrbracket)$ .
3:  $\llbracket t \rrbracket \leftarrow \llbracket x \rrbracket \boxplus 1$ 
4:  $\llbracket t_1 \rrbracket \leftarrow \llbracket t \rrbracket \boxtimes 0.5$ 
5:  $\llbracket t_2 \rrbracket \leftarrow \llbracket t \rrbracket \boxtimes \llbracket t \rrbracket \boxtimes 0.125$ 
6:  $\llbracket y \rrbracket \leftarrow 1 \boxplus \llbracket t_1 \rrbracket \boxplus \llbracket t_2 \rrbracket$ 
7: Return  $\llbracket y \rrbracket$ 

```

$(-5, 0)$. In this stable domain, the term $e^{x/2^\tau}$ is estimated using a 7-degree polynomial, and the final result is subsequently recovered via τ successive squaring operations.

Table 10: Inference accuracy evaluation on GLUE benchmark.

Model	Dataset	Softmax + GELU	GK + ReLU	
		Plaintext	Plaintext	STIP
GPT-2-1.5B	RTE	75.21%	75.36%	75.15%
	SST-2	92.20%	92.01%	91.85%
	QNLI	91.80%	91.13%	90.92%

Table 11: Comparison of an A100 GPU execution time (s) breakdown under LLAMA-3-8B[†] and GPT-2-1.5B[‡].

Operation	STIP			Euston		
	Depth	Time [†]	Time [‡]	Depth	Time [†]	Time [‡]
Input projection	19→10	1.55	0.36	17→16	37.54	5.63
Attention score	10→8	0.69	0.36	16→14	41	16.12
GK / Softmax	8→3	0.13	0.13	14→4	1.69	1.33
Attention output	3→1	0.16	0.08	4→2	40.39	15.71
Bootstrappings	1→8	9.51	4.75	1→10	83.91	32.78
Output projection	8→6	1.25	0.51	2→1	12.51	1.88
LN1	6→1	1.89	0.74	10→1	2.67	1.04
Bootstrappings	1→5	69.91	27.31	1→15	98.73	38.57
FC1	5→4	44.35	7.98	15→14	43.9	7.81
Activation layer	4→2	16.49	7.36	14→2	33.93	15.15
FC2	2→1	43.62	7.82	2→1	43.12	7.74
Bootstrappings	1→10	83.91	32.78	1→10	83.91	32.78
LN2	10→1	2.67	1.04	10→1	2.67	1.04
Bootstrappings	1→19	111	43.31	1→17	108	42.01
Classification	-	7.4	4.4	-	7.4	4.4
Total	-	394.53	138.93	-	641.37	223.99

The homomorphic square root (Algorithm 12) is implemented using a second-order Taylor expansion of the function $\sqrt{1 + (x - 1)}$. Designed for numerical stability within the input range (0, 2), this method approximates $\sqrt{x}$ by computing a weighted summation of the powers of the residual term $(x - 1)$.

I OTHER LLM BENCHMARKS

Table 10 reports the inference accuracy of GPT-2-1.5B. STIP exhibits acceptable deviations from the plaintext baseline. Table 11 reports the end-to-end inference time of STIP on LLAMA-3-8B and GPT-2-1.5B, achieving approximately 1.6× speedup compared to Euston. Note that the order of the first Bootstrappings and Output projection for Euston is adjusted for visual alignment with STIP. In the actual Euston workflow, Output projection precedes the first Bootstrappings.

To further evaluate the model’s performance on the more complex task, we assessed the GPT-2-1.5B model on the LAMBADA (Language Modeling Broadened to Account for Discourse Aspects) dataset [44] after replacing non-linear functions. To comprehensively assess the language modeling capabilities of different non-linear variants, we utilize three metrics [45]. (i) LAMBADA Whole-word Accuracy (LWA) evaluates the exact match accuracy of the final predicted word using a de-tokenized formulation, where higher values indicate better long-range contextual understanding. (ii) Word-level Perplexity (WP) and (iii) Token-level Perplexity (TP) measure the model’s predictive uncertainty normalized at the word

Table 12: Performance comparison of different attention and activation combinations. “↑” indicates higher is better, and “↓” indicates lower is better. Best results for each metric are highlighted in bold.

Architecture Variant	(i) LWA↑	(ii) WP↓	(iii) TP↓
Softmax + GELU	51.14%	8.63	10.63
GK + GELU	52.61%	9.87	6.02
Softmax + ReLU	48.85%	11.47	6.77
GK + ReLU	49.79%	11.31	6.70

and token levels, respectively. For both perplexity metrics, lower values indicate a stronger language modeling capability. The experimental results of the four architectural combinations are summarized in Table 12.

The integration of the GK yields notable improvements in long-range contextual understanding and token-level predictive certainty. Specifically, the “GK + GELU” variant achieves a peak LAMBADA whole-word accuracy of 52.61%, outperforming the baseline’s 51.14%. Moreover, it drastically reduces the token-level perplexity from 10.63 to 6.02. Although a slight increase is observed in word-level perplexity (from 8.63 to 9.87), the overall downstream performance is enhanced. The advantage of the GK lies in its distance-based similarity metric, which provides a naturally bounded and non-negative distribution. This property effectively stabilizes the attention scores and enables the model to map semantically similar keys with higher precision.

Conversely, substituting GELU with ReLU leads to an overall decline in generation capability. The “Softmax + ReLU” variant drops the whole-word accuracy to 48.85% and worsens the word-level perplexity to 11.47. Furthermore, while combining the GK with ReLU slightly mitigates this drop (recovering accuracy to 49.79% and reducing token-level perplexity to 6.70), it remains persistently inferior to the baseline accuracy. This phenomenon stems from the gradient properties of the activation functions. While GELU provides a smooth, non-zero gradient transition around zero that preserves pre-trained feature distributions, the hard zero-gradient of ReLU truncates negative activation features.

Nevertheless, the overall performance of the “GK + ReLU” combination remains highly acceptable. Although its whole-word accuracy experiences a minor degradation compared to the baseline, it successfully maintains an accuracy of 49.79% and achieves a token-level perplexity of 6.70, which is significantly better than the baseline’s 10.63. This indicates that the model preserves its fundamental language generation and reasoning capabilities. Considering the substantial computational acceleration this variant unlocks for secure non-interactive inference, this marginal trade-off in accuracy is completely justified, establishing “GK + ReLU” as a viable and efficient architectural choice.